

Project Sakura 第1卷 基礎編

第1章～第8章 正式版

Project Sakura 第1巻 基礎編 v0.1

第1章 Project Sakuraとは

1.1 はじめに

Project Sakuraは、AIを活用して資産ブログを構築・運営するためのプロジェクトです。

本書では単なるブログの作り方ではなく、GitHubによるバージョン管理、Pythonによる自動化、WordPressとの連携までを一つの流れとして学びます。

1.2 Project Sakuraの目的

Project Sakuraの目的は、AIを活用しながらブログを構築し、そのブログを継続的に育てていくことです。

単に記事を作って公開するだけではなく、

企画

↓

調査

↓

記事作成

↓

画像作成

↓

公開

↓

アクセス確認

↓

改善

という流れを作り、少しずつ運営を効率化していきます。

さらに、Pythonなどを利用して、繰り返し行う作業を自動化していきます。

最終的には、ブログ運営そのものを「仕組み」として構築することを目指します。

1.3 AIを活用する理由

Project

Sakuraでは、AIを単なる文章作成ツールとしてではなく、プロジェクト全体を支援するための道具として利用します。

例えば、

- アイデアの整理
- 情報の整理
- 記事構成の作成
- 文章作成の支援
- タイトル案の作成
- 画像制作の支援
- プログラム作成の支援
- エラーの原因調査
- 作業手順の整理

などにAIを活用します。

ただし、AIが作ったものをそのまま使用するではありません。

内容を確認し、必要に応じて修正し、自分のプロジェクトに適した形へ整えていきます。

AIを「全部やってくれる魔法の箱」として使うのではなく、「作業と一緒に進めるアシスタント」として活用することが重要です。

1.4 Pythonを利用する理由

Project Sakuraでは、Pythonを自動化のための主要なプログラミング言語として利用します。

Pythonは比較的分かりやすい文法を持っており、初心者がプログラミングを学ぶ場合にも適しています。

また、ファイル操作、データ処理、Webサービスとの連携など、さまざまな用途に利用できます。

Project Sakuraでは、最初から難しいプログラムを作ることはしません。

まずは、

```
print("こんにちは！Project Sakuraへようこそ！")
```

という簡単なプログラムから始めます。

このプログラムは、画面に文字を表示するだけの非常に小さなプログラムです。

しかし、プログラミングでは、このような小さな処理を組み合わせることで大きな仕組みを作ることができます。

Project Sakuraでは、

文字を表示する

↓

ファイルを扱う

↓
データを扱う
↓
処理を自動化する
↓
Webサービスと連携する
↓
ブログ運営を支援する

というように、少しずつ発展させていきます。

プログラミング未経験者でも理解できるように、実際に動かしながら学ぶことを基本とします。

1.5 GitHubは何のために使うのか

Project SakuraではGitHubを利用して、プロジェクトのファイルを管理します。

GitHubには、ファイルを保存するだけでなく、変更履歴を残せるという大きな特徴があります。

例えば、

Version 0.1
↓
Version 0.2
↓
Version 0.3

というように、プロジェクトがどのように変化したのかを記録できます。

もし途中でファイルを変更して問題が発生した場合でも、以前の状態を確認したり、変更内容を調べたりできます。

今回作成した、

project-sakura

というリポジトリも、Project Sakuraの開発記録を管理するために利用します。

GitHub Desktopを利用することで、Gitのコマンドを最初から覚えなくても、画面上の操作でファイルの変更を確認し、保存できます。

1.6 現在のProject Sakura

本書の作成開始時点で、基本的な開発環境の準備は完了しています。

現在使用する主な環境は次のとおりです。

```
| 項目 | 状態 |
|---|---|
| GitHubアカウント | 作成完了 |
| GitHub Desktop | インストール完了 |
| VS Code | インストール完了 |
| Python | インストール完了 |
| project-sakura | 作成済み |
| GitHubへの保存 | 確認済み |
| Pythonプログラムの実行 | 確認済み |
| 第1章原稿 | 作成中 |
```

Project Sakuraの基本的なフォルダ構成は、次のように整理していきます。

```
project-sakura
|
|— docs
| |
| |— CHANGELOG.md
| |
| |— Vol01_基礎編
| |
| |— images
| |
| |— source
| |   |— Chapter01.md
| |
| |— README.md
|
|— python
|   |— lesson01.py
|
|— .gitattributes
|
|— README.md
```

1.7 本書の進め方

本書では、最初から高度な知識を必要としません。

基本的には、

説明

↓

実際に操作

↓

結果を確認

↓

問題があれば修正

↓

次へ進む

という流れで進めます。

分からない用語が出てきた場合は、その都度説明します。

操作についても、

「どこをクリックするのか」

「何を入力するのか」

「何が表示されれば成功なのか」

まで、できるだけ具体的に説明します。

本書は、プログラマーだけを対象とした専門書ではありません。

プログラミング未経験者でも、Project Sakuraを少しずつ作れることを目標とします。

1.8 Project Sakuraの最終目標

Project Sakuraの最終目標は、単にブログを1つ作ることはありません。

目指しているのは、

企画

↓

調査

↓

AIによる支援

↓

記事作成

↓

画像作成

↓

公開

↓

アクセス確認

↓

改善

↓

データ蓄積

↓

次の記事へ

という一連の流れを、継続的に回せる仕組みです。

さらに、その仕組み自体も改善していきます。

最初は手作業だった作業を、

手作業

↓

手順化

↓

半自動化

↓

自動化

へと発展させていきます。

これがProject Sakuraの大きなテーマです。

1.9 このプロジェクトで大切にすること

Project Sakuraでは、次の5つを大切にします。

1. 小さく始める

最初から完璧なシステムを作ろうとしません。

小さな機能を一つずつ作ります。

2. 実際に動かして確認する

説明を読むだけではなく、実際に操作して結果を確認します。

3. 失敗も記録する

エラーや失敗も重要な情報です。

同じ問題を繰り返さないために、原因や解決方法を記録します。

4. 作ったものを残す

記事、コード、設計、手順などをGitHubに保存します。

5. 継続できる仕組みにする

一時的に大量の作業をするのではなく、無理なく長期間続けられる仕組みを目指します。

1.10 初心者から始めるということ

Project Sakuraでは、最初から専門知識を持っている必要はありません。

分からないことがあれば、その都度調べます。

エラーが出たら、その原因を確認します。

操作を間違えたら、戻してやり直します。

重要なのは、

「分からないから進めない」

ではなく、

「分からないところを一つずつ解決する」

という進め方です。

プログラムもブログも、一度に完成するものではありません。

小さな完成を積み重ねることで、最終的に大きなシステムになります。

1.11 現在までに行ったこと

Project Sakuraを開始するために、これまでに次の環境を準備しました。

GitHub

GitHubアカウントを作成しました。

Googleアカウントを利用して登録を行い、Project Sakuraの保存場所となるGitHub環境を準備しました。

GitHub Desktop

WindowsでGitHubを扱いやすくするため、GitHub Desktopをインストールしました。

GitHub Desktopを利用することで、コマンドを大量に入力しなくても、

変更されたファイルを確認する

変更内容を確認する

Commitする

GitHubへ送信する

過去の変更履歴を見る

といった操作ができます。

VS Code

プログラムや文章ファイルを編集するため、Visual Studio Codeをインストールしました。

Project Sakuraでは、Pythonプログラムだけではなく、Markdown形式の原稿もVS Codeで編集します。

Python

Pythonをインストールしました。

そして最初のプログラムとして、

```
print("こんにちは！Project Sakuraへようこそ！")
```

を作成し、実際に実行できることを確認しました。

これは小さな一歩ですが、Project Sakuraで初めて動かしたプログラムです。

1.12 ドキュメント管理

Project Sakuraでは、ドキュメントを整理して保存します。

現在は、

```
project-sakura/docs/
```

をドキュメント管理用の場所として使用しています。

第1巻の基礎編については、

```
project-sakura/docs/Vol01_基礎編/
```

の中に原稿を保存します。

さらに、

```
source
```

フォルダには、各章のMarkdown原稿を保存します。

例えば第1章は、

Chapter01.md

として管理します。

このMarkdown原稿を元にして、最終的にはWord版やPDF版などの正式なドキュメントへ展開していきます。

1.13 原稿と正式版の関係

Project Sakuraでは、チャットの内容をそのまま本にするではありません。

まず原稿を作ります。

その原稿を確認します。

必要な修正を行います。

内容が確定した段階で正式版として保存します。

基本的な流れは、

原稿作成

↓

内容確認

↓

修正

↓

章の完成

↓

正式版へ反映

↓

GitHubへ保存

とします。

これにより、Project Sakuraの本そのものも、バージョン管理された成果物として残すことができます。

1.14 Project Sakuraは「本」と「システム」を同時に作る

Project Sakuraには、2つの側面があります。

1つ目は、ブログを運営するためのシステムを作ることです。

2つ目は、そのシステムを作る過程を本として記録することです。

つまり、

ブログ

+

AI

+

Python

+

GitHub

+

WordPress

+

運営ノウハウ

を一つのプロジェクトとしてまとめます。

そのため、本書は単なる操作マニュアルではありません。

Project Sakuraが成長していく過程そのものを記録する「開発記録」であり、「教材」であり、「設計書」でもあります。

1.15 第1章まとめ

Project

Sakuraは、AIを活用しながら資産ブログを構築・運営し、その運営作業も少しずつ仕組み化していくプロジェクトです。

本プロジェクトでは、

AIを活用する

Pythonで自動化する

GitHubで管理する

VS Codeで編集する

WordPressで公開する

データを蓄積する

改善を繰り返す

という流れを作っていきます。

そして、作成したものだけではなく、作業方法や失敗、改善方法についても記録していきます。

Project Sakuraは、

「ブログを作るプロジェクト」

であると同時に、

「ブログを育てる仕組みを作るプロジェクト」

でもあります。

最初から大きなシステムを完成させる必要はありません。

小さなプログラムを動かす。

小さな記事を作る。

小さな改善を行う。

その一つ一つを保存する。

そして、それらを積み重ねていきます。

これがProject Sakuraの基本的な進め方です。

Chapter Check

第1章を読み終えたら、次の質問を確認してみましょう。

Q1

Project Sakuraの目的は何ですか？

Q2

AIはProject Sakuraでどのような役割を担当しますか？

Q3

Pythonは何のために利用しますか？

Q4

GitHubを利用する理由は何ですか？

Q5

Project Sakuraが目指している運営の循環とは何ですか？

Q6

なぜ作った記事やプログラムだけでなく、作業手順や失敗も記録するのでしょうか？

第2章への予告

第1章では、Project Sakuraが何を目標しているのかを確認しました。

第2章からは、いよいよ「何を作るのか」を具体的に設計していきます。

ブログ、AI、Python、GitHub、WordPress、データ管理などが、それぞれどのようにつながるのかを整理します。

そして、Project Sakura全体の構造を明確にします。

第2章では、

「Project Sakura全体設計」

をテーマとして進めます。

Project Sakura 第1巻 基礎編 v0.1

第2章 Project Sakura全体設計

2.1 はじめに

第1章では、Project Sakuraが何を目標しているのかを確認しました。

Project

Sakuraは、AIを活用しながら資産ブログを構築し、その運営作業も少しずつ仕組み化していくプロジェクトです。

第2章では、その目的を実現するために、Project

Sakura全体をどのような仕組みにしていくのかを整理します。

ここでは、いきなり細かいプログラムを作るのではなく、まず全体像を把握します。

大きな仕組みを作るときには、最初に「何を作るのか」「それぞれの部品がどのようにつながるのか」を決めておくことが重要です。

Project Sakuraでは、

企画 ↓ 調査 ↓ AIによる支援 ↓ 記事作成 ↓ 画像作成 ↓ 公開 ↓ アクセス確認 ↓

改善 ↓ データ蓄積

という流れを基本とします。

そして、この流れを繰り返しながら、手作業だった部分を少しずつ仕組み化していきます。

2.2 Project Sakura全体の考え方

Project Sakuraを一言で表すなら、

「ブログ運営を継続できる仕組みを作るプロジェクト」

です。

単純にブログを作るだけなら、WordPressを用意して記事を書けば始めることができます。

しかし、長期間ブログを運営する場合には、それだけでは十分ではありません。

記事のテーマを考える必要があります。

情報を調査する必要があります。

記事を作成する必要があります。

画像を準備する必要があります。

公開後のアクセス状況を確認する必要があります。

さらに、結果を見ながら次の記事を改善する必要があります。

Project Sakuraでは、これらの作業を一つずつ整理します。

そして、

「人が判断する部分」

と

「コンピューターに任せられる部分」

を分けて考えます。

最終的には、人が重要な判断を行い、コンピューターやAIが繰り返し作業を支援する形を目指します。

2.3 Project Sakuraを構成する主な要素

Project Sakuraでは、主に次の要素を利用します。

- AI
- Python
- GitHub
- GitHub Desktop
- VS Code
- WordPress
- データ管理
- 画像作成
- アクセス解析

これらは、それぞれ別々のものとして存在するものではありません。

役割を分担しながら、一つの運営システムとしてつなげていきます。

例えば、

AI ↓ 企画・調査・文章作成を支援

Python ↓ 繰り返し作業やデータ処理を自動化

GitHub ↓ プログラムやドキュメントを管理

VS Code ↓ プログラムや原稿を編集

WordPress ↓ 記事をWeb上へ公開

アクセス解析 ↓ 公開後の結果を確認

という関係になります。

2.4 AIの役割

Project Sakuraでは、AIを重要な支援役として利用します。

ただし、AIにすべてを任せることを目的にはしません。

AIには得意な作業があります。

例えば、

- ・アイデアを出す
- ・情報を整理する
- ・文章の構成を考える
- ・下書きを作る
- ・表現を改善する
- ・プログラム作成を支援する
- ・エラーの原因を調べる
- ・作業手順を整理する

といった作業です。

一方で、最終的な判断が必要な部分もあります。

例えば、

- ・その情報を本当に公開してよいか
- ・内容が正しいか
- ・読者にとって役立つか
- ・Project Sakuraの方針に合っているか
- ・実際のサイトで公開するか

といった判断です。

そのため、Project Sakuraでは、

AIが提案する ↓ 人が確認する ↓ 必要なら修正する ↓ 採用する

という流れを基本とします。

AIは「全部を任せる存在」ではなく、「作業を前に進めるための支援役」として利用します。

2.5 Pythonの役割

Pythonは、Project Sakuraの自動化を担当する重要な要素です。

最初は非常に簡単なプログラムから始めます。

例えば、

```
print("こんにちは！Project Sakuraへようこそ！")
```

というプログラムです。

このプログラムは画面に文字を表示するだけです。

しかし、Pythonでは、このような小さな処理を組み合わせ、より大きな処理を作ることができます。

Project Sakuraでは、今後、

文字を表示する ↓ ファイルを読む ↓ ファイルへ保存する ↓ データを整理する

↓ 繰り返し処理する ↓ Webサービスと通信する ↓ ブログ運営を支援する

という順番で、少しずつPythonの利用範囲を広げていきます。

最初から難しい自動化を作るのではなく、動作を確認しながら段階的に発展させます。

2.6 GitHubの役割

GitHubは、Project Sakuraの「記録庫」として利用します。

GitHubには、プログラムだけではなく、ドキュメントや設計資料なども保存します。

例えば、

- Pythonプログラム
- Markdown原稿
- README
- CHANGELOG
- 設計資料
- 画像
- その他のプロジェクト関連ファイル

などを管理します。

GitHubを利用する大きな理由の一つが、変更履歴を残せることです。

例えば、

Version 0.1 ↓ Version 0.2 ↓ Version 0.3

というように、Project Sakuraがどのように変化してきたのかを確認できます。

間違ってファイルを変更した場合でも、以前の変更内容を確認できます。

Project

Sakuraでは、GitHubを単なる「ファイル置き場」としてではなく、開発記録を残す場所として利用します。

2.7 GitHub Desktopの役割

GitHub Desktopは、GitHubを扱いやすくするためのツールです。

Gitにはコマンドを入力して操作する方法があります。

しかし、Project

Sakuraでは、最初からすべてのGitコマンドを覚える必要はありません。

GitHub Desktopを使えば、

変更されたファイルを確認する ↓ 変更内容を確認する ↓ Commitする ↓

GitHubへ送信する ↓ 履歴を確認する

という作業を画面上で行うことができます。

初心者の段階では、まずGitHub Desktopを利用して、

「ファイルを変更したらCommitする」

という基本的な習慣を身につけます。

2.8 VS Codeの役割

Visual Studio Code、通称VS Codeは、Project Sakuraの主要な編集環境です。

VS Codeでは、

- Python
- Markdown

- その他の設定ファイル

などを編集します。

Project Sakuraでは、プログラムだけではなく、本の原稿もVS Codeで管理します。

例えば第1章は、

docs/Vol01_基礎編/source/Chapter01.md

として保存しています。

第2章は、

docs/Vol01_基礎編/source/Chapter02.md

として保存します。

このように、各章を1つのMarkdownファイルとして管理します。

これにより、章ごとの内容を分かりやすく管理できます。

2.9 WordPressの役割

WordPressは、完成した記事をWeb上へ公開するための中心的な場所として利用します。

Project

Sakuraでは、WordPressを単なるブログ画面としてではなく、記事を公開・管理するためのWebシステムとして考えます。

基本的な流れは、

記事を企画する ↓ 情報を調査する ↓ 記事を作成する ↓ 内容を確認する ↓
画像を準備する ↓ WordPressへ登録する ↓ 公開する ↓ 結果を確認する

となります。

将来的には、Pythonなどを利用して、WordPressとの連携も検討します。

ただし、最初から自動投稿を行うものではありません。

まずは手作業で仕組みを理解し、その後に自動化できる部分を探します。

2.10 データ管理の役割

Project Sakuraでは、データを蓄積することも重要です。

例えば、

- 記事タイトル
- URL
- 投稿日
- カテゴリ
- キーワード
- 記事作成状況
- 更新日
- アクセス状況
- 改善内容

などを記録できます。

最初はExcelやCSV、Markdownなど、扱いやすい形式から始めても構いません。

重要なのは、

「作ったものを記録する」

ことです。

記録が残っていれば、

どの記事を作ったのか ↓ どの記事が読まれたのか ↓ どの記事を改善したのか ↓

次に何を作るのか

という判断につながられます。

2.11 画像作成の役割

ブログ記事では、文章だけではなく画像も重要になります。

Project Sakuraでは、記事の内容に合わせて、

- アイキャッチ画像
- 説明用画像
- 図解
- その他のブログ用素材

などを準備します。

画像についても、記事作成と同じように、

企画 ↓ 作成 ↓ 確認 ↓ 保存 ↓ 公開

という流れで管理します。

画像ファイルについても、必要に応じてGitHubの管理対象として整理します。

2.12 Project Sakuraの基本ワークフロー

Project Sakuraでは、ブログ運営を次のような流れで考えます。

企画
↓
調査
↓
AIによる整理・支援
↓
記事作成
↓
内容確認
↓
画像作成
↓
WordPressへ登録
↓
公開
↓
アクセス確認
↓
改善
↓
データ蓄積
↓
次の企画

この流れを繰り返すことがProject Sakuraの基本です。

重要なのは、記事を1本公開して終わりにしないことです。

公開後の結果を確認し、その結果を次の記事や既存記事の改善に利用します。

2.13 「作る」だけでなく「改善する」

Project Sakuraでは、記事を作ることだけを成功としません。

例えば、記事を公開しても、ほとんど読まれない場合があります。

その場合、

タイトルを変更する ↓ 見出しを改善する ↓ 内容を追加する ↓ 画像を改善する
↓ 内部リンクを見直す ↓ 再度結果を確認する

という改善を行います。

つまり、

作る ↓ 公開する ↓ 確認する ↓ 改善する

というサイクルを回します。

この考え方は、Project Sakura全体に共通します。

2.14 手作業から自動化へ

Project Sakuraでは、最初から完全自動化を目指しません。

まずは手作業で実際の流れを経験します。

例えば記事作成なら、

段階1 手作業

人がすべての作業を行います。

段階2 手順化

作業の順番を決めます。

段階3 半自動化

一部の作業をPythonなどで補助します。

段階4 自動化

繰り返し作業を可能な範囲で自動化します。

この順番を大切にします。

手順を理解しないまま自動化すると、問題が発生したときに原因が分からなくなる可能性があります。

そのため、

「まず自分でやってみる」

ことを基本とします。

2.15 人が担当する部分

Project Sakuraでは、人の判断を重要視します。

例えば、

- ブログの方向性を決める
- 読者を考える
- 記事テーマを決める
- 情報の正確性を確認する
- 公開するか判断する
- 改善方針を決める

といった部分です。

これらは、単純に自動化すればよいものではありません。

Project

Sakuraでは、人が考える部分と、コンピューターに任せる部分を整理します。

2.16 コンピューターに任せる部分

一方で、繰り返し作業には自動化できるものがあります。

例えば、

- ファイル名を整理する
- データを並べ替える
- CSVを読み込む
- 定型的な文章を生成する
- データを集計する
- ファイルをコピーする
- 定型処理を繰り返す

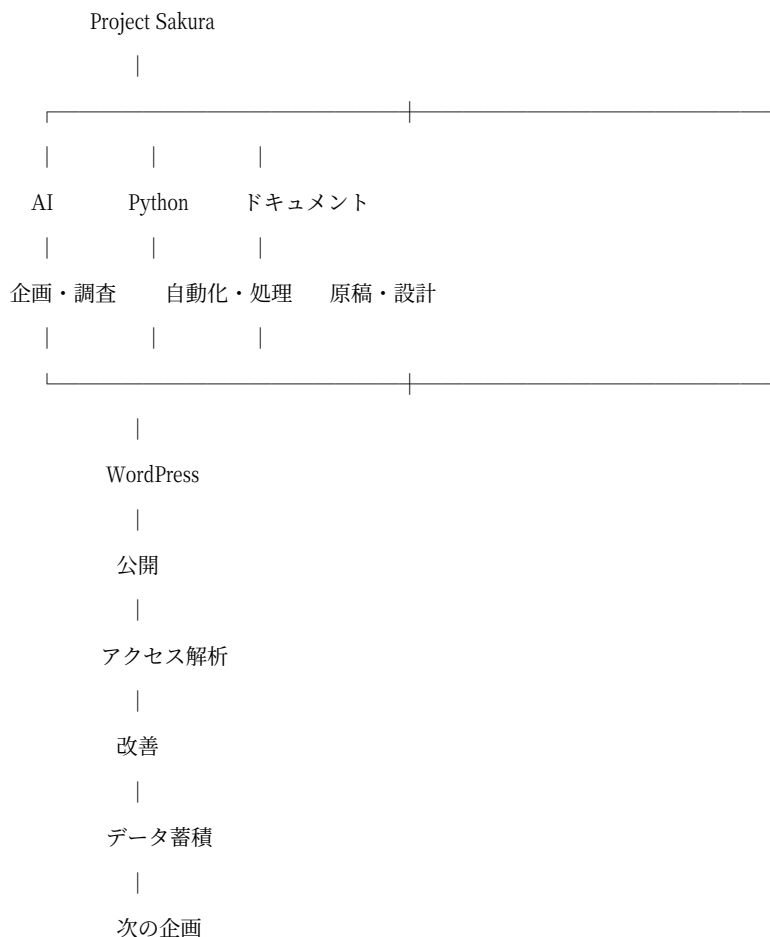
などです。

このような作業はPythonなどを使って自動化できる可能性があります。

ただし、実際に自動化するかどうかは、作業量や安全性を確認した上で判断します。

2.17 Project Sakuraの全体構造

Project Sakuraの全体構造を簡単に表すと、次のようになります。



この構造を基本として、今後少しずつ具体的な機能を追加していきます。

2.18 Project Sakuraのドキュメント構成

Project Sakuraでは、本書そのものもプロジェクトの一部として管理します。

現在の基本構成は次のとおりです。

```
project-sakura
├── docs
│   ├── CHANGELOG.md
│   └── Vol01_基礎編
```



```
|   |— images
|   |
|   |— source
|   |   |— Chapter01.md
|   |   |— Chapter02.md
|   |
|   |— Project_Sakura_Vol1.docx
|   |— Project_Sakura_Vol1.pdf
|   |— README.md
|
|— python
|   |— lesson01.py
|
|— .gitattributes
|
|— README.md
```

第1章、第2章というように、各章をそれぞれ1つのMarkdownファイルとして管理します。

これにより、原稿を修正した場合でも、どの章が変更されたのか分かりやすくなります。

2.19 本書の制作方法

Project

Sakuraでは、本書も通常のプログラム開発と同じように、段階的に作成します。

基本的な流れは、

原稿を作る ↓ 内容を確認する ↓ 修正する ↓ 章を完成させる ↓

GitHubへ保存する ↓ 正式版のWord・PDFへ反映する

という流れです。

各章を個別に管理することで、後から内容を修正しやすくします。

また、正式版のWord・PDFは、確定した原稿から作成します。

2.20 バージョン管理

Project Sakuraでは、文章もプログラムと同じようにバージョンを管理します。

例えば、

v0.1

↓
v0.2
↓
v0.3
↓
v1.0

というように、内容が成長していきます。

最初の段階では、完璧な本を作る必要はありません。

まずはv0.1として完成させます。

その後、内容を追加したり、説明を改善したりしながら、次のバージョンへ進めます。

2.21 エラーや失敗も資産にする

Project Sakuraでは、失敗も記録します。

例えば、

- ファイルの保存場所を間違えた
- Pythonが実行できなかった
- GitHubへの保存方法が分からなかった
- PDFを作成したが表示できなかった
- Markdownの書式が崩れた

といった問題が発生することがあります。

これらは無駄ではありません。

問題が発生したときに、

何が起きたのか ↓ 原因は何だったのか ↓ どうやって解決したのか ↓

次回どうすればよいのか

を記録しておけば、今後の作業に役立ちます。

Project Sakuraでは、こうした経験も開発記録として残します。

2.22 初心者でも進められる設計

Project Sakuraでは、初心者でも作業を進められることを重視します。

そのため、難しい機能を一気に作るのではなく、

小さな作業 ↓ 動作確認 ↓ 理解 ↓ 次の作業

という流れを繰り返します。

例えばPythonなら、

文字を表示する ↓ 変数を使う ↓ 条件分岐を使う ↓ 繰り返し処理を使う ↓

ファイルを扱う ↓ データを扱う ↓ Webと連携する

というように段階的に進めます。

分からない部分があれば、その部分まで戻って確認します。

2.23 Project Sakuraの成長イメージ

Project Sakuraは、最初は非常に小さなプロジェクトです。

最初に作ったPythonプログラムも、画面に文字を表示するだけでした。

しかし、

小さなプログラム ↓ 小さな自動化 ↓ 複数の処理 ↓ ブログ運営支援 ↓

データ管理 ↓ 運営システム

というように、少しずつ大きくしていきます。

重要なのは、最初から大きなものを作ろうとしないことです。

一つの小さな機能を完成させ、それを積み重ねます。

2.24 第2章まとめ

Project Sakuraは、

AI + Python + GitHub + VS Code + WordPress + データ管理

などを組み合わせて、ブログ運営を継続的に改善する仕組みを作るプロジェクトです。

それぞれの役割を整理すると、

AI → 企画・調査・文章作成などを支援する

Python → 繰り返し作業やデータ処理を自動化する

GitHub → プログラムや原稿、変更履歴を管理する

VS Code → プログラムやMarkdown原稿を編集する

WordPress → 記事をWeb上へ公開する

データ管理 → 作ったものや結果を記録する

という関係になります。

そして、これらを一つの流れとしてつなげます。

企画

↓

調査

↓

AI支援

↓

記事作成

↓

画像作成

↓

公開

↓

アクセス確認

↓

改善

↓

データ蓄積

↓

次の企画

Project Sakuraでは、この循環を少しずつ改善していきます。

最初は手作業でも構いません。

作業を理解し、手順を整理し、その後で自動化します。

この考え方を基本として、次の章から具体的な設計へ進みます。

Chapter Check

第2章を読み終えたら、次の質問を確認してみましょう。

Q1

Project

Sakuraでは、なぜブログを作るだけでなく、運営の仕組みまで作ろうとしているのでしょうか？

Q2

AIはProject Sakuraでどのような役割を担当しますか？

Q3

PythonはProject Sakuraで何を自動化するために利用しますか？

Q4

GitHubを利用する主な理由は何ですか？

Q5

VS Codeは何を編集するために利用しますか？

Q6

WordPressはProject Sakuraの中でどのような役割を担当しますか？

Q7

なぜ最初から完全自動化を目指さないのでしょうか？

Q8

Project Sakuraの基本的な運営サイクルを説明してください。

第3章への予告

第2章では、Project

Sakuraを構成する主要な要素と、それぞれの役割を整理しました。

次の章では、さらに具体的な設計へ進みます。

ブログの目的、読者、ブログのテーマ、カテゴリ、記事構成、キーワード、コンテンツ管理、収益化、運営方法などについて整理します。

Project

Sakuraという「仕組み」を実際に動かすためには、まず「何を発信するブログなのか」を明確にする必要があります。

第3章では、

「ブログ設計とコンテンツ戦略」

をテーマとして、Project Sakuraのブログ部分を具体的に設計していきます。

Project Sakura 第1巻 基礎編 v0.1

第3章 ブログ設計とコンテンツ戦略

第2章では、Project Sakuraを構成する主要な要素と、それぞれの役割を整理しました。

Project Sakuraでは、

企画

↓

調査

↓

AI支援

↓

記事作成

↓

画像作成

↓

公開

↓

アクセス確認

↓

改善

↓

データ蓄積

↓

次の企画

という循環を基本とします。

この循環を実際に動かすためには、まず「何を発信するブログなのか」を明確にする必要があります。

ブログを作るだけなら、WordPressを用意して記事を書き始めることはできます。

しかし、長期間にわたってブログを育てていくのであれば、「誰に向けて」「何を」「どのような形で」「どのような目的で」「どのように改善しながら」発信するのかを決めておく必要があります。

本章では、Project Sakuraのブログ部分について、基本的な設計を行います。

ここで決めた内容は、今後の記事作成、AIの利用、画像作成、WordPressでの公開、データ管理、収益化、改善作業につながっていきます。

3.1 ブログ設計を最初に行う理由

ブログの記事を1本書くだけであれば、細かな設計は必要ないかもしれません。

しかし、Project Sakuraが目指しているのは、記事を1本だけ作ることはありません。

記事を作り、公開し、結果を確認し、改善し、次の記事を作る。この作業を継続することが目的です。

そのため、記事を書く前に、ブログ全体の方向性を決めておくことが重要になります。

記事のテーマが毎回大きく変わると、ブログ全体として何を扱っているのか分かりにくくなります。

反対に、ある程度テーマを整理しておけば、「この記事はどのカテゴリに入るのか」「次にどんな記事を書くべきか」「過去の記事とどのようにつながるのか」を考えやすくなります。

Project Sakuraでは、記事単体ではなく、ブログ全体を一つの仕組みとして考えます。

3.2 ブログの目的を決める

最初に決めるのは、ブログを運営する目的です。

Project Sakuraでは、ブログを長期的に育てていくことを基本とします。

そのため、目的は単純に「記事をたくさん公開すること」ではありません。

記事を作る。読者に届ける。公開後の結果を見る。必要に応じて改善する。その結果を次の記事作成に利用する。この循環を続けることが重要です。

「この記事は何のために作るのか」「読者にどのような情報を届けるのか」「公開後に何を確認するのか」という質問に答えられる状態を目指します。

3.3 読者を考える

次に考えるのが、誰に向けたブログなのかという点です。

同じテーマの記事でも、読者によって必要な説明は変わります。専門知識を持っている人に向ける記事と、初めてそのテーマを知る人に向ける記事では、説明の順番や用語の使い方が異なります。

Project Sakuraでは、「何を知りたいのか」「どこで迷うのか」「どこまで説明すれば理解できるのか」を考えることを基本とします。

読者を考えることは、記事を書くためだけではありません。カテゴリ、記事タイトル、画像を考えるときにも関係します。

3.4 ブログのテーマを決める

読者を考えたら、次にブログのテーマを整理します。

ここで重要なのは、最初から完璧なテーマを決めることはありません。

Project Sakuraでは、実際に記事を作り、公開し、結果を確認しながら改善していくことを重視します。

最初に大まかな方向性を決め、記事を作る、反応を確認する、必要ならテーマを調整する、という進め方を取ります。

テーマを決めるときには、継続して扱えるか、読者にとって役立つ内容にできるか、複数の記事へ発展できるか、調査や更新を継続できるか、といった点を確認します。

3.5 カテゴリを設計する

ブログのテーマが決まったら、記事を整理するためのカテゴリを考えます。

カテゴリは記事を分類するためだけのものではありません。ブログ全体の構造を分かりやすくする役割もあります。

記事が増えても管理できるように、必要なカテゴリを作り、記事が増えてきた段階で整理します。最初からカテゴリを増やしすぎないことも重要です。

カテゴリはブログの成長に合わせて見直していきます。

3.6 記事を「単独のページ」として考えない

Project Sakuraでは、一つの記事だけを完成させれば終わりとは考えません。記事同士のつながりも重要です。

基本を説明する記事、応用を説明する記事、実践方法を説明する記事、関連する問題を解決する記事、というように複数の記事をつなげることができます。

記事を企画するときには、「この1本の記事を書く」だけでなく、「このテーマについて、今後どのような記事を作れるか」まで考えるようにします。

3.7 記事の基本構成

Project Sakuraでは、記事の構成もある程度整理しておきます。

基本的な考え方として、

タイトル

↓

導入

↓

本文

↓

まとめ

という流れを基本形とします。

必要に応じて、見出し、説明、具体例、手順、注意点、まとめなどを組み合わせます。

すべての記事を同じ形にすることではなく、記事の目的に合わせて構成を変えながらも、読者が内容を追やすい形にすることが重要です。

3.8 キーワードを考える

記事を企画するときには、読者がどのような言葉で情報を探すのかを考えます。これがキーワードを考えるという作業です。

例えば、「〇〇 方法」「〇〇 使い方」「〇〇 初心者」「〇〇 比較」など、目的に応じて検索する言葉が変わります。

Project Sakuraでは、「この記事が必要としている人は、どのような言葉で情報を探すのか」を考えます。

キーワードは記事タイトル、見出し、本文、関連コンテンツなどを考える際の参考になります。

ただし、キーワードを無理に文章へ詰め込むことが目的ではありません。最も重要なのは、読者が必要としている情報を分かりやすく提供することです。

3.9 コンテンツ管理

記事を増やしていくと、どの記事を作ったのか分からなくなる可能性があります。そこで、Project Sakuraでは記事の情報を記録します。

第2章で整理したように、記事タイトル、URL、投稿日、カテゴリ、キーワード、記事作成状況、更新日、アクセス状況、改善内容などを記録できます。

最初から高度なデータベースを作る必要はありません。Excel、CSV、Markdownなど、扱いやすい形式から始めても構いません。

重要なのは、作ったものを記録することです。

3.10 記事の状態を管理する

記事には、作成途中のものもあれば、公開済みのものもあります。そのため、記事の状態を管理できるようにします。

例えば、

企画中

↓

調査中

↓

構成作成中

↓

執筆中

↓

確認中

↓

画像準備中

↓

公開準備完了

↓

公開済み

↓

改善中

という状態を設定できます。

このようにしておけば、現在どこまで進んでいるのかを確認できます。複数の記事を同時に扱う場合でも、作業の途中で迷いにくくなります。

最初は手作業で管理し、実際の運用を理解した後に、Pythonなどを利用した効率化を検討します。

3.11 AIを使った記事制作

Project Sakuraでは、AIの記事制作の支援に利用します。

AIに任せる作業としては、記事のアイデア整理、構成案の作成、情報の整理、文章の下書き、表現の改善、関連する項目の洗い出しなどが考えられます。

ただし、AIが作った文章をそのまま公開することを基本とはしません。

AIが作成する

↓

人が確認する

↓

必要な情報を追加する

↓

表現を修正する

↓

公開する

という流れを基本とします。

AIは記事を作るための道具であり、最終的な内容を確認する役割は人間が担当します。

3.12 調査と内容確認

記事を作成する場合、文章を書くことだけが重要なわけではありません。記事の内容が正しいか確認することも重要です。

Project Sakuraでは、企画、調査、AIによる整理・支援、記事作成、内容確認、という流れを基本とします。

特に、数字、サービスの仕様、料金、制度、日付など、変更される可能性がある情報については注意します。

AIが提示した情報であっても、そのまま正しいとは限りません。必要な場合には、元となる情報を確認し、公開前に内容を確認します。

3.13 画像とコンテンツの関係

ブログでは、文章だけではなく画像も利用します。

第2章では、アイキャッチ画像、説明用画像、図解、その他のブログ用素材などを準備する方針を整理しました。

画像についても、企画、作成、確認、保存、公開、という流れで管理します。

「この記事には、どのような画像が必要なのか」を記事設計の段階から考えます。

3.14 WordPressで公開する

作成した記事は、最終的にWordPressで公開します。

第2章で整理した基本的な流れは、記事を企画する、情報を調査する、記事を作成する、内容を確認する、画像を準備する、WordPressへ登録する、公開する、結果を確認する、というものです。

WordPressは、単なる文章を書く場所ではありません。Project Sakuraでは、記事を公開・管理するためのWebシステムとして扱います。

将来的には、Pythonなどを利用してWordPressとの連携も検討します。

ただし、最初から自動投稿を行うものではありません。まず手作業で仕組みを理解し、その後に自動化できる部分を探します。

3.15 収益化を考える

Project Sakuraでは、資産ブログを長期的に育てることを目指しています。そのため、収益化についてもブログ設計の段階から考えておきます。

ただし、収益だけを目的にして記事を作るではありません。

まず、読者に役立つ情報を提供する、記事を継続的に公開する、アクセスや反応を確認する、改善する、という基本を優先します。

その上で、ブログの内容や読者に合った収益化方法を検討します。

収益化の方法については、今後の運営状況を見ながら具体化していきます。この段階では、収益化をブログ運営の一部として位置付け、記事の品質や継続性を優先します。

3.16 公開後にアクセスを確認する

記事を公開したら、そこで作業が終了するわけではありません。Project Sakuraでは、公開後の結果を確認します。

例えば、どの記事が読まれたのか、どの記事が読まれなかったのか、どの記事から次の記事へ進んだのか、どのような検索から訪問されたのか、どの記事を改善する必要があるのか、などを確認します。

最初からすべてを高度に分析する必要はありません。まずは、確認できる情報を記録するところから始めます。

3.17 改善する

アクセス状況や記事の内容を確認したら、必要に応じて改善します。

例えば、タイトルを見直す、見出しを整理する、説明を追加する、画像を改善する、古くなった情報を更新する、関連する記事へのリンクを追加する、などの改善が考えられます。

改善した内容も記録します。

これにより、何を変更したのか、変更後にどうなったのか、次に何を改善するのか、を確認できるようになります。

3.18 コンテンツ戦略を「積み上げ」として考える

Project Sakuraでは、記事を単発の作業として扱いません。

一つの記事を作る。その記事から関連するテーマを見つける。次の記事を作る。さらに関連する記事を作る。このように、記事を少しずつ積み上げていきます。

その結果として、記事、関連する記事、カテゴリ、ブログ全体、という構造ができていきます。

最初から大量の記事を作る必要はありません。小さな記事を作り、それを確認し、改善し、次につなげることを基本とします。

3.19 Project Sakuraのコンテンツ制作サイクル

ここまでの内容を整理すると、Project Sakuraのコンテンツ制作は次のようになります。

企画

↓

読者を考える

↓

テーマを決める

↓

キーワードを考える

↓

情報を調査する

↓

記事構成を作る

↓

AIの支援を利用する

↓

記事を書く

↓

内容を確認する

↓

画像を準備する

↓

WordPressへ登録する

↓

公開する

↓

アクセスを確認する

↓

改善する

↓

データを記録する

↓

次の企画へ進む

この流れを繰り返すことで、ブログを少しずつ育てていきます。

3.20 手作業から自動化へ

Project Sakuraでは、最初からすべてを自動化しません。

第2章で整理したように、まずは作業を理解し、手順を整理します。その後、手作業、手順化、半自動化、自動化、という順番で発展させます。

例えば、記事管理について、最初は表に手入力する、Pythonで一覧を読み込む、記事の状態を確認する、必要な情報を自動整理する、というように少しずつ仕組み化できます。

重要なのは、自動化することそのものではありません。「どの作業を自動化すれば、継続しやすくなるのか」を考えることです。

3.21 この章で決めた基本方針

第3章では、Project Sakuraのブログ部分について、次の基本方針を整理しました。

1. ブログの目的を明確にする

何のためにブログを運営するのかを考えます。

2. 読者を考える

誰に向けて情報を発信するのかを考えます。

3. テーマを整理する

継続的に記事を作れるテーマを考えます。

4. カテゴリを整理する

記事が増えても管理できる構造を作ります。

5. 記事をつなげる

単独の記事ではなく、関連する記事を積み上げます。

6. キーワードを考える

読者がどのような言葉で情報を探すのかを考えます。

7. 記事の状態を管理する

企画から公開、改善までの進行状況を記録します。

8. AIを支援役として利用する

AIを活用しながら、人が内容を確認します。

9. 画像も管理する

記事と画像を一つのコンテンツとして考えます。

10. 公開後も改善する

公開を終了地点ではなく、次の改善につながる地点として扱います。

11. データを残す

作ったもの、結果、改善内容を記録します。

12. 必要な部分から自動化する

手作業を理解した後で、Pythonなどを利用して効率化します。

3.22 第3章まとめ

Project Sakuraでは、ブログを単に記事を掲載する場所として考えません。

ブログは、企画、調査、AI支援、記事作成、画像作成、公開、アクセス確認、改善、データ蓄積、次の企画、という循環の中心となります。

そのため、ブログを作る前に、「何を発信するのか」「誰に届けるのか」「どのように記事を整理するのか」「どのように記事を作るのか」「公開後に何を確認するのか」を考えておくことが重要です。

また、Project Sakuraでは、最初から完璧なシステムを作ることを目指しません。

まず手作業で実際に運営します。作業を理解します。問題点を見つけます。手順を整理します。その後で、必要な部分を自動化します。

この順番で進めることで、単に動くプログラムを作るのではなく、実際のブログ運営に役立つ仕組みを作っていきます。

第3章で整理したブログ設計は、今後の記事作成、AI活用、画像作成、WordPress運営、データ管理、自動化の基礎になります。

Chapter Check

第3章を読み終えたら、次の質問を確認してみましょう。

Q1

なぜProject Sakuraでは、記事を書く前にブログ全体を設計するのでしょうか？

Q2

ブログの目的を明確にすることには、どのような意味がありますか？

Q3

読者を考えることは、記事作成以外にどのような部分へ影響しますか？

Q4

カテゴリを作る目的は何ですか？

Q5

なぜ記事を単独のページとしてではなく、関連する記事の集合として考えるのでしょうか？

Q6

キーワードを考えるときに、最も重要なことは何でしょうか？

Q7

AIの記事制作に利用するとき、人が確認する必要があるのはなぜでしょうか？

Q8

Project Sakuraでは、なぜ最初から記事作成やWordPress投稿を完全自動化しないのでしょうか？

Q9

公開後にアクセス状況を確認する目的は何ですか？

Q10

Project Sakuraが目指す「手作業から自動化へ」の流れを説明してください。

第4章への予告

第3章では、Project Sakuraのブログ部分について、「何を発信するのか」「誰に届けるのか」「記事をどのように整理するのか」「どのように制作し、公開し、改善するのか」という基本設計を整理しました。

次の段階では、この設計を実際の運営に落とし込んでいきます。

記事を作るためには、テーマを決めるだけでは足りません。

どのように情報を集めるのか。どのように記事の構成を作るのか。AIに何を依頼するのか。人間がどこを確認するのか。どのような手順で記事を完成させるのか。

こうした具体的な制作手順を決める必要があります。

第4章では、

****「記事制作ワークフロー」****

をテーマとして、Project Sakuraで実際に記事を1本作るための流れを具体化していきます。

Project Sakura 第1巻 基礎編 v0.1

第4章 記事制作ワークフロー

第3章では、Project Sakuraのブログについて、「何を発信するのか」「誰に届けるのか」「記事をどのように整理するのか」「どのように制作し、公開し、改善するのか」という基本設計を整理しました。

第4章では、その設計を実際の作業へ落とし込みます。

Project Sakuraでは、ブログ運営を、

企画

↓

調査

↓

AIによる整理・支援

↓

記事作成

↓

内容確認

↓

画像作成

↓

WordPressへ登録

↓

公開

↓

アクセス確認

↓

改善

↓

データ蓄積

↓

次の企画

という循環として考えます。

この流れは、第2章で整理したProject Sakuraの基本ワークフローそのものです。

本章では、この一連の流れを「記事を1本完成させる」という視点から、順番に確認します。

最初からすべてを自動化する必要はありません。

まずは一つの記事を手作業で完成させます。

その作業を理解し、手順を整理します。

その後で、時間のかかる作業や繰り返し作業を見つけ、必要な部分から少しずつ仕組み化していきます。

4.1 記事制作を一つの流れとして考える

記事を書くとなると、「文章を書く」ことだけを想像するかもしれません。

しかし、Project Sakuraでは、記事制作を文章を書く作業だけとは考えません。

一つの記事を公開するまでには、

テーマを決める

↓

情報を集める

↓

構成を考える

↓

記事を書く

↓

内容を確認する

↓

画像を準備する

↓

WordPressへ登録する

↓

公開する

という複数の作業があります。

さらに、公開した後は、

アクセスを確認する

↓

必要な部分を改善する

↓

結果を記録する

という作業があります。

そのため、Project Sakuraでは「記事を書く」ではなく、「記事を完成させて公開し、その後の改善まで行う」ことを一つの制作サイクルとして扱います。

4.2 最初に記事の目的を確認する

記事を作る前に、その記事を作る目的を確認します。

第3章では、ブログの目的や読者、テーマを考えることが重要だと整理しました。

そのため、記事ごとに、「この記事は何のために作るのか」を確認します。

例えば、

読者の疑問を解決する

↓

必要な情報を分かりやすく説明する

↓

関連する記事へ案内する

というように、記事の役割を決めます。

目的が決まっていれば、記事の構成を考えるとにも判断しやすくなります。

文章を書くこと自体が目的にならないようにすることが重要です。

4.3 記事のテーマを決める

目的を確認したら、記事のテーマを決めます。

第3章では、ブログ全体のテーマと、そこから継続的に記事を作れることが重要だと整理しました。

一つの記事についても、「このテーマについて何を説明するのか」を具体化します。

テーマが広すぎる場合は、いくつかの記事に分けます。

逆に、内容が少なすぎる場合は、関連する内容と組み合わせることも考えます。

最初から完璧な答えを出す必要はありません。

記事を作りながら、どこまで説明するのが適切なのかを確認していきます。

4.4 読者が知りたいことを整理する

テーマが決まったら、その記事を読む人が何を知りたいのかを考えます。

読者が初心者なのか、ある程度知識を持っているのかによって、必要な説明は変わります。

そのため、

「読者は何に困っているのか」

「何を知りたいのか」

「どこで迷うのか」

を整理します。

記事の内容を考えると、自分が説明したいことだけではなく、読者が必要としている情報を基準にします。

4.5 キーワードを整理する

読者が情報を探すときに使う言葉も確認します。

第3章では、キーワードは記事タイトル、見出し、本文、関連コンテンツなどを考える際の参考になると整理しました。

ここでは、「この記事が必要としている人は、どのような言葉で情報を探すのか」を考えます。

キーワードを整理することは、単に検索される言葉を並べるためではありません。

読者が何を求めているのかを考えるための材料として利用します。

4.6 情報を調査する

テーマと読者を整理したら、必要な情報を調査します。

Project Sakuraでは、

企画

↓

調査

↓

AIによる整理・支援

↓

記事作成

↓

内容確認

という順番を基本とします。

文章を書く前に必要な情報を集めておくことで、記事の構成を考えやすくなります。

調査では、

- 基本的な情報
- 具体例
- 必要な手順
- 注意点
- 関連する情報

などを整理します。

ただし、集めた情報をすべて記事に入れる必要はありません。

記事の目的と読者に必要な情報を選びます。

4.7 情報の正確性を確認する

調査した情報については、内容が正しいか確認します。

特に、

数字

サービスの仕様

料金

制度

日付

など、変更される可能性がある情報には注意します。

AIが提示した情報についても、そのまま正しいとは限りません。

そのため、

情報を調べる

↓

内容を整理する

↓

必要に応じて元となる情報を確認する

↓

記事に利用する

という考え方を基本とします。

Project Sakuraでは、AIを便利な支援役として利用しますが、最終的な内容確認は人間が担当します。

4.8 記事の構成を作る

情報を集めたら、いきなり本文を書き始めるのではなく、まず構成を作ります。

基本的な構成として、

タイトル

↓

導入

↓

本文

↓

まとめ

を考えます。

必要に応じて、

見出し

↓

説明

↓

具体例

↓

手順

↓

注意点

などを組み合わせます。

構成を先に作っておけば、文章を書くときに内容が途中で大きく脱線しにくくなります。

また、情報の抜けや重複も確認しやすくなります。

4.9 AIに支援してもらう

構成ができたら、AIを利用して記事制作を支援します。

Project Sakuraでは、AIに、

- 記事のアイデア整理
- 構成案の作成
- 情報の整理
- 文章の下書き
- 表現の改善

- ・関連する項目の洗い出し

などを支援してもらいます。

ここで重要なのは、AIにすべてを任せることではありません。

AIが作成する

↓

人が確認する

↓

必要な情報を追加する

↓

表現を修正する

↓

公開する

という流れを基本とします。

AIは記事制作を支援する道具です。

記事の内容に責任を持ち、最終的に公開する文章を確認するのは人間です。

4.10 本文を作成する

構成と必要な情報が整理できたら、本文を作成します。

本文では、読者が最初から最後まで内容を追えるようにします。

基本的には、

導入

↓

問題やテーマの説明

↓

具体的な内容

↓

必要な手順

↓

注意点

↓

まとめ

という流れを意識します。

ただし、すべての記事を同じ構成にする必要はありません。

記事の目的に合わせて構成を変えます。

重要なのは、読者が「今、何について説明されているのか」を理解できることです。

4.11 初心者向けの記事では説明を省略しない

Project Sakuraでは、初心者にも理解できる記事を作ることを大切にします。

そのため、専門用語を使う場合には、必要に応じて説明します。

例えば、

「これは何なのか」

「なぜ必要なのか」

「どこを操作するのか」

「何が表示されれば成功なのか」

まで説明することを意識します。

知識がある人にとっては当たり前のことでも、初心者にとっては重要な情報になる場合があります。

読者が途中で止まらないように、必要なところでは説明を追加します。

4.12 記事を確認する

本文が完成したら、公開前に確認します。

確認する項目として、

- 記事の目的と内容が一致しているか
- 必要な情報が入っているか
- 説明が分かりやすいか
- 誤った情報がないか
- 古い情報が混ざっていないか
- 文章に不自然な部分がないか
- 見出しの順番が分かりやすいか

などを確認します。

特にAIを利用した記事では、人間による確認を必ず行います。

AIが作成した文章だから正しい、とは考えません。

4.13 記事の状態を管理する

記事を複数作るとなると、どの記事がどこまで進んでいるのかを管理する必要があります。

Project Sakuraでは、例えば、

企画中

↓

調査中

↓

構成作成中

↓

執筆中

↓

確認中

↓

画像準備中

↓

公開準備完了

↓

公開済み

↓

改善中

という状態を設定できます。

この状態を記録しておけば、作業の途中で、「今どこまで進んでいたのか」が分からなくなることを防げます。

最初は手作業で管理します。

実際の運用を理解した後で、Pythonなどを利用した効率化を検討します。

4.14 画像を準備する

記事の内容が固まったら、必要な画像を準備します。

第2章では、

- ・アイキャッチ画像
- ・説明用画像
- ・図解

- ・その他のブログ用素材

を準備する方針を整理しました。

画像も記事と同じように、

企画

↓

作成

↓

確認

↓

保存

↓

公開

という流れで管理します。

重要なのは、画像の記事の最後に付け足すだけにしないことです。

記事を設計する段階から、「この記事にはどのような画像が必要なのか」を考えます。

文章だけでは分かりにくい内容については、説明用画像や図解を利用します。

4.15 WordPressへ登録する

記事と画像の準備が完了したら、WordPressへ登録します。

Project Sakuraでは、WordPressを単なるブログ画面ではなく、記事を公開・管理するためのWebシステムとして考えます。

基本的な登録作業として、

記事本文を登録する

↓

タイトルを設定する

↓

カテゴリを設定する

↓

画像を設定する

↓

必要な情報を確認する

↓

公開する

という流れを取ります。

最初は手作業で行います。

手作業で一連の流れを経験することで、どの作業に時間がかかるのか、どこに間違いが起きやすいのかを確認できます。

4.16 公開前の最終確認

WordPressへ登録した後も、公開前に最終確認を行います。

確認する内容は、

- ・タイトル
- ・本文
- ・見出し
- ・画像
- ・カテゴリ
- ・リンク
- ・表示内容
- ・必要な情報

などです。

記事の原稿上では問題なくても、Webページとして表示したときに問題が見つかる場合があります。

そのため、公開ボタンを押す前に、最終確認を行います。

4.17 公開する

確認が完了したら、記事を公開します。

ここで一つの記事制作サイクルが一区切りになります。

しかし、Project Sakuraでは、公開を最終地点とは考えません。

公開した後には、

アクセス確認

↓

改善

↓

データ蓄積

という次の作業があります。

つまり、「公開したら終了」ではなく、「公開したところから次の改善が始まる」と考えます。

4.18 公開後のアクセス確認

記事を公開したら、結果を確認します。

例えば、

どの記事が読まれたのか

どの記事が読まれなかったのか

どの記事から次の記事へ進んだのか

どのような検索から訪問されたのか

などを確認します。

最初から高度な分析を行う必要はありません。

まずは確認できる情報を記録します。

結果を記録することで、次の記事を作るときの判断材料になります。

4.19 記事を改善する

アクセス状況や記事の内容を確認したら、必要に応じて改善します。

例えば、

タイトルを見直す

↓

説明を追加する

↓

見出しを整理する

↓

画像を改善する

↓

古くなった情報を更新する

↓

関連する記事へのリンクを追加する

などの改善が考えられます。

改善した内容も記録します。

何を変更したのか。

変更後にどうなったのか。

次に何を改善するのか。

これらを記録しておけば、ブログ運営を感覚だけに頼らず、少しずつ改善できます。

4.20 データを記録する

Project Sakuraでは、「作ったものを記録する」ことを重要視します。

記事については、

- 記事タイトル
- URL
- 投稿日
- カテゴリ
- キーワード
- 記事作成状況
- 更新日
- アクセス状況
- 改善内容

などを記録できます。

最初はExcel、CSV、Markdownなど、扱いやすい形式から始めても構いません。

重要なのは、作った記事と、その結果を残すことです。

記録が残っていれば、

どの記事を作ったのか

↓

どの記事が読まれたのか

↓

どの記事を改善したのか

↓

次に何をやるのか

という判断につなげることができます。

4.21 記事制作の一連の流れ

ここまでの内容をまとめると、Project Sakuraで記事を1本作る流れは次のようになります。

企画

↓

記事の目的を確認

↓

テーマを決める

↓

読者を考える

↓

キーワードを整理する

↓

情報を調査する

↓

情報の正確性を確認する

↓

記事構成を作る

↓

AIによる整理・支援

↓

本文を作成する

↓

内容を確認する

↓

画像を準備する

↓

WordPressへ登録する

↓

公開前に最終確認する

↓

公開する

↓

アクセスを確認する

↓

改善する

↓

データを記録する

↓

次の記事を企画する

この流れを一つの基本手順として扱います。

4.22 最初は手作業で行う

Project Sakuraでは、この一連の作業を最初から自動化しません。

理由は、実際の作業を理解することが重要だからです。

例えば、

記事の構成を作る

↓

AIに支援してもらう

↓

文章を確認する

↓

WordPressへ登録する

という作業を実際に行えば、

「ここは時間がかかる」

「ここは何度も同じ操作をしている」

「ここは間違いやすい」

という問題が見えてきます。

問題が分かれば、自動化する場所も判断できます。

そのため、

手作業

↓

手順化

↓

半自動化

↓

自動化

という順番に進めます。

4.23 Pythonによる効率化につなげる

Project Sakuraでは、Pythonを利用して作業を少しずつ効率化していきます。

例えば、

記事データを読み込む

↓

記事の状態を一覧表示する

↓

必要な情報を整理する

↓

繰り返し作業を補助する

といった処理をPythonで作ることができます。

ただし、どの作業を自動化するのかは、実際の運用を経験した後で判断します。

最初から複雑なプログラムを作るではありません。

まず人間が作業を行います。

その作業を記録します。

そして、繰り返し発生する作業を見つけます。

その後でPythonによる効率化を検討します。

4.24 AIと人間の役割を分ける

Project Sakuraでは、AIと人間の役割を分けて考えます。

AIは、

情報整理

↓

アイデア整理

↓

構成案

↓

下書き

↓

表現改善

↓

関連項目の整理

などを支援します。

人間は、

目的の確認

↓

情報の確認

↓

内容の判断

↓

必要な修正

↓

最終確認

↓

公開

を担当します。

AIを使うこと自体が目的ではありません。

AIを利用することで、記事制作を効率化しながら、最終的な品質を確認できる仕組みを作ることが目的です。

4.25 一つの記事から次の記事へつなげる

記事を公開して終わりにしないことも重要です。

一つの記事を作ると、そのテーマから新しい記事のアイデアが見つかる場合があります。

例えば、

基本を説明する記事

↓

応用を説明する記事

↓

実践方法を説明する記事

↓

よくある問題を解決する記事

というように発展できます。

さらに、公開後のアクセス状況から、「読者が次に何を知りたいのか」を考えることもできます。

このように、一つの記事を次の記事の企画につなげます。

4.26 記事制作を継続できる仕組みにする

Project Sakuraの目的は、一時的に大量の記事を作ることではありません。

長期間にわたって、無理なく記事を作り続けられる仕組みを作ることです。

そのためには、

記事を企画する

↓

記事を作る

↓

公開する

↓

結果を確認する

↓

改善する

↓

記録する

↓

次の記事を企画する

という循環を維持する必要があります。

作業量が大きくなったら、手順を見直します。

繰り返し作業が増えたら、自動化できる部分を探します。

このようにして、ブログ運営そのものを少しずつ改善します。

4.27 第4章で決めた基本方針

第4章では、記事制作について次の基本方針を決めました。

1. 記事を作る前に目的を確認する
2. 読者が必要としている情報を考える
3. キーワードを整理する
4. 必要な情報を調査する

5. 情報の正確性を確認する
6. 本文を書く前に構成を作る
7. AIは記事制作の支援役として利用する
8. AIが作った内容を人間が確認する
9. 記事の状態を管理する
10. 記事に必要な画像を準備する
11. WordPressへ登録する
12. 公開前に最終確認する
13. 公開後のアクセスを確認する
14. 必要に応じて記事を改善する
15. 作ったものと結果を記録する
16. 最初は手作業で行い、その後に自動化する

4.28 第4章まとめ

Project Sakuraの記事制作は、単に文章を書く作業ではありません。

企画

↓

調査

↓

AIによる整理・支援

↓

記事作成

↓

内容確認

↓

画像作成

↓

WordPressへ登録

↓

公開

↓

アクセス確認

↓

改善

↓

データ蓄積

↓

次の企画

という一連の流れとして考えます。

この流れを実際に行うことで、どこに時間がかかるのか、どこで問題が起きるのか、どの作業を改善できるのかが分かります。

そのため、Project Sakuraでは最初から完全自動化を目指しません。

まず手作業で実際に運営します。

作業を理解します。

手順を整理します。

問題を記録します。

そして、必要な部分から少しずつ自動化します。

AIは記事制作を支援し、Pythonは繰り返し作業の効率化を支援します。

WordPressは記事を公開・管理する場所として利用します。

そして、GitHubなどを利用して、作ったものや作業記録を残します。

一つの記事を完成させることは、小さな作業に見えるかもしれませんが。

しかし、その一つの記事を作る流れを整理し、記録し、改善できるようにすることが、Project Sakura全体の仕組み作りにつながります。

Chapter Check

第4章を読み終えたら、次の質問を確認してみましょう。

Q1

Project Sakuraでは、なぜ記事制作を「文章を書く作業」だけとは考えないのでしょうか？

Q2

記事を作る前に、記事の目的を確認するのはなぜですか？

Q3

記事を作成する前に情報を調査する目的は何ですか？

Q4

AIが作成した文章を、そのまま公開しないのはなぜですか？

Q5

本文を書く前に記事構成を作るメリットは何ですか？

Q6

記事の状態を管理すると、どのようなことが分かるようになりますか？

Q7

画像の記事制作のどの段階から考えるべきでしょうか？

Q8

WordPressへ登録した後、公開前に最終確認を行うのはなぜですか？

Q9

公開後にアクセスを確認する目的は何ですか？

Q10

Project Sakuraでは、なぜ最初から完全自動化を目指さないのでしょうか？

第5章への予告

第4章では、Project Sakuraで記事を1本完成させるための基本的なワークフローを整理しました。

次の段階では、このワークフローを実際の作業環境へ落とし込んでいきます。

記事を作るためには、原稿だけではなく、

記事データ

画像

作業状態

更新履歴

関連資料

制作記録

なども管理する必要があります。

また、Project Sakuraでは、これらの情報を後からPythonなどで扱えるようにしていきます。

そのためには、最初に「どこへ、何を保存するのか」を決めておく必要があります。

第5章では、

「Project Sakuraのファイル・データ管理」

をテーマとして、原稿、画像、プログラム、データ、ドキュメントなどをどのように整理していくかを具体化します。

そして、GitHub上のProject Sakuraを、単なるファイル置き場ではなく、プロジェクト全体の記録を管理する場所として、さらに具体的に活用していきます。

Project Sakura 第1巻 基礎編 v0.1

第5章 Project Sakuraのファイル・データ管理

第4章では、Project Sakuraで記事を1本完成させるための基本的なワークフローを整理しました。

企画

↓

調査

↓

AIによる整理・支援

↓

記事作成

↓

内容確認

↓

画像作成

↓

WordPressへ登録

↓

公開

↓

アクセス確認

↓

改善

↓

データ蓄積

↓

次の企画

という流れです。

しかし、記事を作る数が増えてくると、文章を書くだけでは管理できなくなります。

記事の原稿、画像、Pythonプログラム、設定ファイル、作業記録、設計書など、さまざまなファイルが増えていくからです。

そこで第5章では、Project Sakuraで扱うファイルやデータをどのように整理するのかを決めます。

最初から複雑なデータベースを作る必要はありません。

まずは「どこに何を保存するのか」を分かりやすく決めます。

そして、将来的にPythonなどから扱いやすい構造へ少しずつ発展させていきます。

5.1 なぜファイル管理が必要なのか

Project Sakuraを始めたばかりの段階では、ファイルの数はそれほど多くありません。

例えば、

Chapter01.md

Chapter02.md

lesson01.py

程度であれば、どこにあるのかすぐに分かります。

しかし、プロジェクトが成長すると、

Chapter01.md

Chapter02.md

Chapter03.md

Chapter04.md

Chapter05.md

article001.md

article002.md

article003.md

image001.png

image002.png

image003.png

というように、ファイルが増えていきます。

さらに、

Pythonプログラム

設定ファイル

CSV

JSON

Word

PDF

画像

資料

なども加わります。

何でも一つのフォルダへ保存してしまうと、必要なファイルを探すだけで時間がかかります。

そのため、Project Sakuraでは、ファイルの種類や役割ごとに保存場所を分けます。

5.2 Project Sakuraの基本フォルダ構成

Project Sakuraでは、基本的に次のような構造で管理します。

```
project-sakura
|
|— docs
| |
| |— CHANGELOG.md
| |
| |— Vol01_基礎編
| |
| |— images
| |
| |— source
| |   |— Chapter01.md
| |   |— Chapter02.md
| |   |— Chapter03.md
| |   |— Chapter04.md
| |   |— Chapter05.md
| |
| |— README.md
|
|— python
|   |— lesson01.py
|
|— .gitattributes
|
|— README.md
```

この構造は、ファイルを見たときに「これは何のためのファイルなのか」が分かることを目的としています。

5.3 docsフォルダ

`docs` は、Project Sakuraのドキュメントを保存する場所です。

例えば、

設計書

教材

操作手順

開発記録

各巻の原稿

などを保存します。

Project Sakuraでは、ブログ運営システムそのものだけではなく、その作り方や考え方も記録します。

そのため、ドキュメント管理はプロジェクトの重要な部分です。

5.4 Vol01_基礎編フォルダ

第1巻の基礎編に関するファイルは、

`docs/Vol01\_基礎編/`

にまとめます。

ここには、第1巻に関係する原稿や画像、READMEなどを保存します。

第1巻が完成した後も、内容を更新する可能性があります。

そのため、章ごとの原稿と正式版のファイルを整理して保存します。

5.5 sourceフォルダ

`source` は、正式版を作るための元原稿を保存する場所です。

例えば、

Chapter01.md

Chapter02.md

Chapter03.md

Chapter04.md

Chapter05.md

というように、各章をMarkdownファイルとして保存します。

Project Sakuraでは、これらのMarkdown原稿を基本となる原稿として扱います。

この方法には大きなメリットがあります。

原稿をテキストとして管理できるため、VS Codeで編集しやすく、GitHubで変更履歴も確認できます。

さらに、後からWordやPDFへ変換することもできます。

5.6 なぜMarkdownを使うのか

Project Sakuraでは、章の原稿にMarkdownを利用します。

Markdownは、特別なソフトがなくても比較的簡単に編集できる文章形式です。

例えば、

``# 第5章``

と書けば大きな見出しとして扱うことができます。

``## 5.1 なぜファイル管理が必要なのか``

と書けば、その下の見出しとして整理できます。

Markdownの良いところは、文章そのものと構造が一緒に管理できることです。

また、GitHubとの相性も良く、変更履歴を確認しやすくなります。

Project Sakuraでは、Markdownを「本の元原稿」として利用します。

5.7 正式版と元原稿を分ける

Project Sakuraでは、

元原稿

↓

確認

↓

完成

↓

Word / PDF

という流れで正式版を作ります。

そのため、Markdown原稿とWord/PDFを同じ役割のファイルとして考えません。

Markdownは編集しやすい元原稿です。

Wordは読みやすい文書として利用します。

PDFは配布・閲覧しやすい正式版として利用します。

それぞれ役割が違います。

5.8 imagesフォルダ

`images` フォルダには、ドキュメントやブログ制作で利用する画像を保存します。

例えば、

アイキャッチ画像

説明用画像

図解

スクリーンショット

操作説明用画像

などです。

画像が増えると、ファイル名だけでは何の画像なのか分からなくなる場合があります。

そのため、画像についても、できるだけ分かりやすい名前を付けます。

例えば、

chapter05_folder_structure.png

のように、用途が分かる名前にします。

5.9 ファイル名を分かりやすくする

ファイル名は、後から見たときに内容が分かるようにします。

例えば、

test.png

だけでは、何の画像なのか分かりません。

一方、

chapter05_folder_structure.png

なら、第5章のフォルダ構成に関する画像だと分かります。

同じように、

article_001.md

chapter05.md

など、一定のルールを決めておくと管理しやすくなります。

ファイル名のルールは、プロジェクトが大きくなるほど重要になります。

5.10 Pythonフォルダ

`python` フォルダには、Project Sakuraで利用するPythonプログラムを保存します。

現在は学習用の小さなプログラムから始めています。

例えば、

lesson01.py

のようなファイルです。

今後、Pythonを利用して、

記事データの整理

↓

ファイルの確認

↓

データの集計

↓

繰り返し作業の補助

などを行えるように発展させます。

最初からブログ全体を自動化するものではありません。

小さなプログラムを作り、動作を確認しながら少しずつ発展させます。

5.11 データとプログラムを分ける

Pythonのプログラムと、プログラムが扱うデータは、できるだけ役割を分けます。

例えば、

python

↓

処理を行うプログラム

data

↓

処理対象となるデータ

という考え方です。

現段階では必要なフォルダだけを作り、プロジェクトが大きくなった段階でデータ用フォルダを追加していきます。

重要なのは、「プログラムの中にすべてのデータを書き込む」という構造にしないことです。

データと処理を分けておくと、後から変更しやすくなります。

5.12 CSVとは何か

Project Sakuraでは、将来的に記事や作業データをCSV形式で扱うことも考えます。

CSVは、表形式のデータを保存するためのシンプルな形式です。

例えば、

```
記事ID,タイトル,状態
001,Python入門,公開済み
002,GitHub入門,執筆中
003,WordPress入門,企画中
```

のように、行と列で情報を管理できます。

Excelなどでも開くことができます。

最初は手作業で管理していても、データ量が増えたらCSVなどを利用することで、Pythonから扱いやすくなります。

5.13 JSONとは何か

JSONも、Project Sakuraで将来的に利用する可能性があるデータ形式です。

JSONは、設定や構造化されたデータを保存するのに向いています。

例えば、

```
{
  "title": "Python入門",
  "status": "published"
}
```

のように情報を保存できます。

CSVは表形式のデータに向いています。

JSONは、項目が増えたり、階層構造を持ったデータを扱ったりするときに便利です。

どちらを使うかは、保存するデータの種類によって判断します。

5.14 データ形式は目的に合わせる

Project Sakuraでは、すべてのデータを一つの形式で保存する必要はありません。

文章

→ Markdown

表形式のデータ

→ CSV

設定や構造化データ

→ JSON

プログラム

→ Python

正式な文書

→ Word / PDF

というように、用途に応じて使い分けます。

重要なのは、ファイル形式そのものを覚えることではありません。

「この情報は何なのか」

「誰が使うのか」

「後からどのように利用するのか」

を考えて保存形式を決めることです。

5.15 GitHubで管理する理由

Project Sakuraでは、作ったファイルをGitHubで管理します。

GitHubを利用することで、ファイルの変更履歴を残すことができます。

例えば、

第1版

↓

修正版

↓

第2版

↓

正式版

という変化を記録できます。

これは文章だけではなく、Pythonプログラムや設定ファイルにも利用できます。

何を変更したのかを確認できるため、プロジェクトが大きくなっても過去の状態を追跡できます。

5.16 Commitとは何か

GitHubを利用すると、「Commit」という言葉が出てきます。

Commitは、簡単に言えば、

「今回行った変更を、一つの記録として保存する」

という操作です。

例えば、

Chapter05.mdを追加した

↓

変更内容を確認する

↓

Commitする

という流れです。

Commitメッセージには、何を変更したのか分かる内容を書きます。

例えば、

`Add Chapter05`

のように記録します。

後から履歴を見たときに、何をしたのか分かりやすくなります。

5.17 Pushとは何か

Commitしただけでは、変更内容がGitHub上のリポジトリに送信されていない場合があります。

そこでPushを行います。

イメージとしては、

ファイルを変更

↓

Commit

↓

Push

↓

GitHubへ反映

という流れです。

GitHub Desktopを利用すれば、画面上の操作で行うことができます。

最初は用語が分かりにくいかもしれません。

しかし、

Commit = 変更を記録する

Push = GitHubへ送る

と覚えておけば十分です。

5.18 GitHubに何でも保存すればよいわけではない

GitHubは便利ですが、すべてのファイルを無条件に保存する場所ではありません。

特に、

パスワード

APIキー

秘密情報

個人情報

認証情報

などは、公開リポジトリへ保存しないように注意します。

Project Sakuraでは、GitHubへ保存するファイルと、保存してはいけない情報を区別します。

今後、外部サービスと連携する場合には、この点が特に重要になります。

5.19 .gitignoreについて

Gitでは、特定のファイルを管理対象から外すために`.gitignore`という仕組みを利用できます。

例えば、

一時ファイル

個人設定

秘密情報を含むファイル

実行時に生成されるファイル

などをGitの管理対象から外すことができます。

Project Sakuraでも、必要に応じて`.gitignore`を設定します。

ただし、設定を理解せずに追加するのではなく、「なぜ除外するのか」を確認しながら利用します。

5.20 README.mdの役割

Project Sakuraには、

`README.md`

を用意します。

READMEは、そのフォルダやプロジェクトについて説明するためのファイルです。

例えば、

このプロジェクトは何なのか

↓

何が入っているのか

↓

どうやって使うのか

↓

現在どこまで進んでいるのか

などを説明できます。

初めてプロジェクトを見る人にとって、READMEは案内板のような役割を持ちます。

5.21 CHANGELOG.mdの役割

Project Sakuraでは、変更履歴を整理するために、

`CHANGELOG.md`

も利用します。

例えば、

2026-XX-XX

- ・第5章を追加
- ・フォルダ構成を整理

のように、プロジェクトに行った大きな変更を記録します。

Gitの詳細な履歴とは別に、人間が読みやすい形で変更点をまとめることが目的です。

5.22 記事データを管理する

記事が増えてきたら、記事そのもののだけではなく、記事に関する情報も管理します。

例えば、

記事ID

タイトル

カテゴリ

キーワード

状態

公開日

更新日

URL

などです。

最初はMarkdownや表計算ソフトなどで管理しても構いません。

重要なのは、記事の状態を把握できることです。

例えば、

企画中

↓

執筆中

↓

確認中

↓

公開済み

という状態を管理できます。

5.23 記事IDを付ける

記事数が増えると、タイトルだけで管理するのが難しくなります。

そこで、記事にIDを付ける方法があります。

例えば、

ART-001

ART-002

ART-003

のように番号を付けます。

記事タイトルが変更されても、IDは変えないようにします。

そうすることで、記事を管理するための固定された識別子として利用できます。

ただし、最初から複雑なルールを作る必要はありません。

必要になった段階で導入します。

5.24 作業状態を管理する

記事には、

企画中

調査中

執筆中

確認中

公開準備

公開済み

改善中

などの状態があります。

これを管理することで、「今どの記事を作業すべきなのか」が分かります。

将来的にはPythonを使って、

公開準備まで進んでいる記事

↓

確認が必要な記事

↓

更新時期が来た記事

などを一覧表示することもできます。

このような機能は、Project Sakuraの自動化につながっていきます。

5.25 ファイル管理からデータ管理へ

最初は「ファイルを整理する」ことから始めます。

しかし、Project Sakuraが成長すると、単なるファイル管理だけでは足りなくなります。

例えば、

記事

↓

記事の状態

↓

記事のURL

↓

アクセス情報

↓

更新履歴

↓

関連画像

↓

関連キーワード

というように、複数の情報を関連付けて管理する必要が出てきます。

これが「データ管理」です。

ファイル管理からデータ管理へ発展させることで、後からPythonなどによる処理がしやすくなります。

5.26 最初からデータベースを作らない

データ管理という言葉を知ると、すぐにデータベースを作る必要があるように感じるかもしれません。

しかし、Project Sakuraでは、最初から大きなデータベースを作りません。

まず、

何のデータが必要なのか

↓

どのように使うのか

↓

どのくらい増えるのか

を確認します。

実際に運用してから必要な仕組みを作ります。

この考え方は、第4章で決めた、

手作業

↓

手順化

↓

半自動化

↓

自動化

という方針と同じです。

5.27 バックアップについて

重要なファイルについては、バックアップも考えます。

GitHubで管理しているからといって、すべての問題がなくなるわけではありません。

また、ローカルPC上のファイルが誤って削除される可能性もあります。

そのため、

GitHub

+

ローカル環境

+

必要に応じたバックアップ

というように、複数の保存先を考えます。

ただし、バックアップ先にも秘密情報が含まれないように注意します。

5.28 フォルダを増やしすぎない

整理するときに注意したいのが、フォルダを増やしすぎることです。

例えば、

docs

↓

book

↓

vol01

↓

basic

↓

chapters

↓

source

↓

chapter

↓

draft

↓

final

というように、細かく分けすぎると、逆に探しにくくなります。

最初は、

「見れば分かる」

程度のシンプルな構造を目指します。

必要になったら、後から分割します。

5.29 フォルダ構成も改善する

Project Sakuraのフォルダ構成は、最初に決めたものを永遠に固定する必要はありません。

実際に運用して、

「ここは分かりにくい」

「このファイルが増えすぎた」

「このデータは別にした方がいい」

という問題が出てきたら改善します。

フォルダ構成そのものも、Project Sakuraの成長に合わせて進化させます。

5.30 VS Codeでファイルを管理する

VS Codeでは、左側のエクスプローラーからフォルダやファイルを確認できます。

例えば、

docs

python

などのフォルダを開いて、必要なファイルを選択します。

Project Sakuraでは、VS Codeを、

原稿を書く

↓

Pythonを書く

↓

ファイルを確認する

↓

フォルダ構成を確認する

ための中心的な編集環境として利用します。

5.31 GitHub Desktopで変更を確認する

ファイルを変更したら、GitHub Desktopを開きます。

すると、変更されたファイルが一覧に表示されます。

例えば、

Chapter05.md

を追加した場合、そのファイルが変更として表示されます。

そこで、

変更されたファイルを確認する

↓

変更内容を確認する

↓

Commitする

↓

Pushする

という流れでGitHubへ保存します。

この操作を繰り返すことで、Project Sakuraの開発記録が残っていきます。

5.32 「保存」と「GitHubへの保存」は違う

初心者のうちは、ここが混乱しやすいポイントです。

VS Codeで、

`Ctrl + S`

を押すと、現在のファイルがPC上に保存されます。

これは「ファイルを保存した」状態です。

その後、

GitHub Desktop

↓

Commit

↓

Push

を行うことで、GitHub側にも変更が反映されます。

つまり、

Ctrl + S

↓

PCに保存

Commit

↓

変更を記録

Push

↓

GitHubへ送信

という違いがあります。

この3つを区別できるようになることは、Project Sakuraを進める上で重要です。

5.33 作業記録を残す

Project Sakuraでは、完成したファイルだけではなく、作業記録も重要です。

例えば、

何を作ったのか

↓

どこでエラーが出たのか

↓

どう解決したのか

↓

何を変更したのか

を記録します。

これは将来、自分自身が同じ問題に遭遇したときにも役立ちます。

また、Project Sakuraそのものが「開発記録」であるため、失敗も含めて記録することに価値があります。

5.34 初心者だからこそ記録する

初心者の段階では、分からないことがたくさん出てきます。

しかし、それは問題ではありません。

むしろ、

「何が分からなかったのか」

「どこでつまづいたのか」

「どう解決したのか」

を記録しておけば、後から非常に価値のある教材になります。

Project Sakuraでは、初心者が実際に作りながら学んでいく過程そのものも記録します。

5.35 第5章で決めた基本方針

第5章では、ファイル・データ管理について次の基本方針を決めました。

1. ファイルを役割ごとに整理する
2. 第1巻の原稿は `docs/Vol01\_基礎編/source/` に保存する
3. 各章は一つのMarkdownファイルとして管理する

4. 画像はimagesフォルダで管理する
5. Pythonプログラムはpythonフォルダで管理する
6. Markdownを本の元原稿として利用する
7. WordとPDFは正式版として扱う
8. CSVやJSONは用途に応じて利用する
9. GitHubで変更履歴を管理する
10. CommitとPushの役割を理解して使う
11. 秘密情報をGitHubへ保存しない
12. READMEとCHANGELOGを活用する
13. 記事データの状態を管理する
14. 必要になった段階でPythonによるデータ処理へ発展させる
15. 最初から複雑なデータベースを作らない
16. フォルダ構成は実際の運用に合わせて改善する
17. 作業内容や失敗も記録する

5.36 第5章まとめ

Project Sakuraが成長すると、文章だけではなく、画像、プログラム、データ、設定、作業記録など、多くのファイルを扱うようになります。

そのため、最初の段階から「どこに何を保存するのか」を整理しておくことが重要です。

Project Sakuraでは、

docs

↓

ドキュメント

source

↓

Markdown原稿

images

↓

画像

python

↓

Pythonプログラム

というように、役割ごとにファイルを整理します。

また、

Markdown

↓

元原稿

Word

↓

文書版

PDF

↓

正式版

というように、ファイル形式ごとの役割も分けます。

さらに、GitHubを利用して変更履歴を残し、CommitとPushによって作業内容を保存します。

記事数が増えれば、記事そのものだけではなく、

記事ID

タイトル

状態

URL

公開日

更新日

アクセス情報

などのデータも管理する必要があります。

ただし、最初から複雑なシステムを作る必要はありません。

まずは実際にファイルを作り、実際に記事を作り、実際に運用します。

そこで発生した問題を記録します。

そして、必要になったところからデータ管理やPythonによる自動化へ発展させます。

Project Sakuraでは、

小さく作る

↓

実際に使う

↓

問題を見つける

↓

整理する

↓

改善する

↓

自動化する

という考え方を繰り返します。

ファイル管理も、その一つです。

最初は小さなフォルダ構成でも、ルールを守って整理していけば、Project Sakuraが大きくなったときにも管理しやすくなります。

そして、その整理されたデータが、将来的なPythonによる自動化や、ブログ運営全体の仕組み化につながっていきます。

Chapter Check

第5章を読み終えたら、次の質問を確認してみましょう。

Q1

Project Sakuraでファイルを整理する必要があるのはなぜですか？

Q2

`docs` フォルダには、どのようなファイルを保存しますか？

Q3

`source` フォルダにMarkdown原稿を保存する理由は何ですか？

Q4

Markdown、Word、PDFには、それぞれどのような役割がありますか？

Q5

CSVはどのようなデータを扱うのに向いていますか？

Q6

JSONはどのような用途で利用できますか？

Q7

CommitとPushには、それぞれどのような役割がありますか？

Q8

GitHubへ保存してはいけない情報には、どのようなものがありますか？

Q9

記事IDや記事の状態を管理するメリットは何ですか？

Q10

Project Sakuraでは、なぜ最初から複雑なデータベースを作らないのでしょうか？

Q11

VS Codeで `Ctrl + S` を押すことと、GitHubへ保存することにはどのような違いがありますか？

Q12

Project Sakuraで、作業中の失敗やエラーも記録するのはなぜですか？

第6章への予告

第5章では、Project Sakuraで扱うファイルやデータを、どのように整理して管理するのかを確認しました。

ここまでで、

Project Sakuraとは何か

↓

Project Sakura全体の構造

↓

ブログをどのように設計するか

↓

記事をどのように制作するか

↓

ファイルやデータをどのように管理するか

という基本的な土台ができました。

第6章からは、いよいよProject Sakuraの中心的な道具の一つである**AIの活用方法**について、より具体的に考えていきます。

AIに何を任せるのか。

人間は何を判断するのか。

どのような指示を出せばよいのか。

AIが作った内容をどのように確認するのか。

そして、AIを使うことで記事制作やブログ運営をどのように効率化できるのか。

第6章では、

「Project SakuraにおけるAI活用」

をテーマとして進めます。

AIを「全部やってくれる魔法の道具」として扱うのではなく、Project Sakuraと一緒に作っていくための「支援ツール」として、具体的な使い方を整理していきます。

Project Sakura 第1巻 基礎編 v0.1

第6章 Project SakuraにおけるAI活用

6.1 はじめに

第5章までで、Project Sakuraの基本的な土台を整理しました。

Project Sakuraとは何か。

全体をどのような仕組みにするのか。

ブログをどのように設計するのか。

記事をどのように制作するのか。

そして、原稿や画像、プログラム、データをどのように管理するのか。

ここまでの章では、Project Sakuraを長く続けるための基本構造を作ってきました。

第6章では、いよいよProject Sakuraの中心的な道具の一つであるAIについて、具体的に考えていきます。

第1章でも説明したように、Project SakuraではAIを単なる文章作成ツールとして利用するわけではありません。

AIは、

- ・アイデアの整理
- ・情報の整理
- ・記事構成の作成
- ・文章作成の支援
- ・タイトル案の作成
- ・画像制作の支援
- ・プログラム作成の支援
- ・エラーの原因調査
- ・作業手順の整理

など、さまざまな場面で利用します。

ただし、AIが作ったものをそのまま使用するものではありません。

内容を確認し、必要に応じて修正し、自分のプロジェクトに適した形へ整えていきます。

Project Sakuraでは、AIを「全部やってくれる魔法の箱」として扱うのではなく、「作業と一緒に進めるアシスタント」として利用します。

この考え方を、第6章ではもう少し具体的に整理していきます。

6.2 AIは何のために使うのか

AIを使う目的を一言で表すなら、

「作業を効率化しながら、考えるための支援を受ける」

ことです。

AIにすべてを任せることが目的ではありません。

例えば、記事を書く場合でも、

テーマを決める

↓

情報を整理する

↓

記事構成を考える

↓

文章を書く

↓

内容を確認する

↓

修正する

↓

公開する

という流れがあります。

この中のすべてをAIに任せる必要はありません。

むしろ、

「どこをAIに任せると効率が良くなるのか」

を考えることが重要です。

例えば、最初のアイデアを整理したり、複数の案を比較したり、文章の構成を考えたりする作業では、AIが役立つ場合があります。

一方で、Project Sakuraとして本当に採用するかどうか、公開してよい内容なのか、自分のブログの目的に合っているか、といった判断は人間が行います。

6.3 AIと人間の役割を分ける

Project Sakuraでは、AIと人間の役割を分けて考えます。

AIが得意なことと、人間が判断すべきことは同じではありません。

例えばAIには、

情報を整理する

↓

複数の案を出す

↓

文章を作る

↓

構成を提案する

↓

表現を変える

といった支援をしてもらいます。

人間は、

目的を決める

↓

採用する案を決める

↓

内容を確認する

↓

正しいか判断する

↓

公開するか決める

という役割を担当します。

このように役割を分けることで、AIを便利に使いながら、Project Sakuraそのものの方向性を人間が管理できます。

6.4 AIに丸投げしない

AIを使い始めると、

「全部作ってください」

と依頼したくなるかもしれません。

しかし、Project Sakuraでは、最初からすべてをAIに丸投げする方法は採用しません。

理由は単純です。

AIが作ったものが、

Project Sakuraの目的に合っているのか。

読者にとって役立つのか。

内容に問題がないのか。

実際の運営に使えるのか。

ということ、人間が確認する必要があるからです。

また、自分自身が内容を理解しないままAIに任せてしまうと、問題が発生したときに原因を確認できなくなります。

Project Sakuraでは、

AIに作ってもらう

↓

人間が確認する

↓

必要なら修正する

↓

完成させる

という流れを基本とします。

6.5 AIを使う前に目的を決める

AIに質問する前に、まず「何をしたいのか」を決めます。

例えば、

「ブログ記事を書きたい」

だけでは、まだ少し曖昧です。

より具体的に、

「初心者向けにPythonの基本を説明する記事の構成を作りたい」

とすれば、AIも目的を理解しやすくなります。

さらに、

誰向けなのか

↓

何を説明したいのか

↓

どの程度の難しさなのか

↓

どのような形式にしたいのか

などを伝えると、より具体的な結果を得やすくなります。

Project Sakuraでは、AIに質問する前に「目的」を整理することを大切にします。

6.6 良い指示とは何か

AIに出す指示は、一般に「プロンプト」と呼ばれます。

プロンプトとは、AIに対して、

「何をしてほしいのか」

を伝えるための指示です。

例えば、

「Pythonの記事を書いて」

だけでもAIは文章を作れます。

しかし、Project Sakuraでは、もう少し条件を具体化します。

例えば、

「Python未経験者向けの記事構成を作ってください。専門用語はできるだけ簡単に説明し、実際にプログラムを動かしながら学べる構成にしてください。」

というようにします。

このように、

目的

↓

対象

↓

条件

↓

出力してほしいもの

を伝えることで、AIとのやり取りを整理しやすくなります。

6.7 プロンプトの基本構造

Project Sakuraでは、最初は次のような形で考えると分かりやすいでしょう。

目的

↓

誰向けか

↓

何をしてほしいか

↓

条件

↓

出力形式

例えば、

目的：

初心者向けPython記事を作る

対象：

プログラミング未経験者

依頼：

記事構成を作る

条件：

専門用語を分かりやすく説明する

出力：

見出しと各見出しの説明

という形です。

すべてのプロンプトをこの形式にしなければならないわけではありません。

しかし、AIへの依頼内容を整理する考え方として非常に役立ちます。

6.8 最初から完璧なプロンプトを作らない

初心者のうちは、完璧なプロンプトを作ろうとする必要はありません。

まず簡単な指示を出します。

結果を確認します。

「ここが足りない」

「ここは難しすぎる」

「もっと具体的にしてほしい」

という部分があれば、追加で指示します。

つまり、

指示

↓

結果

↓

確認

↓

追加指示

↓

改善

という会話を繰り返します。

Project Sakuraでは、この繰り返しを重要な作業として扱います。

AIとのやり取りは、一回の質問ですべてを完成させるものではありません。

6.9 AIに記事構成を作ってもらう

記事を書くとき、いきなり本文を書き始めると、途中で内容が重複したり、必要な説明が抜けたりすることがあります。

そこで、最初に記事構成を作ります。

例えば、

タイトル

↓

導入

↓

基礎知識

↓

具体例

↓

実際の手順

↓

注意点

↓

まとめ

というように、記事全体の流れを決めます。

AIには、この構成案を作る支援をしてもらえます。

ただし、構成案をそのまま採用する必要はありません。

Project Sakuraのブログ設計や読者に合っているかを確認します。

6.10 AIに文章作成を支援してもらおう

構成が決まったら、AIに本文作成を支援してもらうことができます。

例えば、

「この見出しについて初心者向けに説明してください」

という依頼を出します。

AIが文章を作ったら、

内容を確認する

↓

分かりにくい部分を確認する

↓

事実関係を確認する

↓

Project Sakuraの文章として整える

という作業を行います。

ここで重要なのは、AIが作った文章を完成品として扱わないことです。

AIは文章作成を支援する道具です。

最終的な原稿を完成させる作業は、人間が確認しながら行います。

6.11 AIが作った情報を確認する

AIを利用するときに特に重要なのが確認です。

AIは自然な文章を作ることができます。

しかし、自然な文章であることと、内容が正しいことは同じではありません。

そのため、

数字

↓

日付

↓

製品名

↓

サービス仕様

↓

プログラムの動作

↓

法律や規約などの重要情報

など、確認が必要な情報については、元の情報を確認します。

Project Sakuraでは、

AIが言ったから正しい

とは考えません。

必要な情報について確認し、問題がないことを確認してから利用します。

6.12 AIの回答をそのまま引用しない

AIに質問すると、詳しい説明が返ってくることがあります。

しかし、その文章をそのまま記事に貼り付けるのではなく、自分のブログの目的に合わせて整理します。

例えば、

AIの回答

↓

必要な情報を確認

↓

重要な部分を整理

↓

自分の記事構成に合わせる

↓

文章を確認

↓

公開用原稿にする

という流れです。

こうすることで、AIとの会話とProject Sakuraの正式な原稿を区別できます。

6.13 AIによるアイデア整理

AIは、記事を書く前のアイデア整理にも利用できます。

例えば、

「このテーマについて、記事にできそうな切り口を10個出してください」

と依頼します。

すると複数の案を得られます。

その中から、

ブログの目的に合うか

↓

読者に必要か

↓

既存記事と重複しないか

↓

実際に調査できるか

などを確認します。

重要なのは、AIが10個案を出したから10個全部採用するのではないことです。

AIは選択肢を増やすために利用し、採用するかどうかは人間が判断します。

6.14 AIによるタイトル案作成

記事タイトルもAIに案を出してもらえます。

例えば、

「初心者向けPython記事のタイトル案を10個作ってください」

という依頼です。

複数の候補を比較することで、タイトルを考える時間を短縮できます。

ただし、

内容とタイトルが一致しているか

↓

誇張していないか

↓

読者に分かりやすいか

↓

Project Sakuraのブログに合っているか

を確認します。

タイトルだけを強くして、本文の内容が追いつかない状態にはしません。

6.15 AIによる文章の改善

AIは、新しく文章を書くときだけでなく、すでに書いた文章を改善する場合にも利用できます。

例えば、

「この文章を初心者にも分かりやすい表現にしてください」

と依頼できます。

また、

「重複している部分を見つけてください」

「見出しの順番を確認してください」

「専門用語が難しい部分を指摘してください」

といった使い方もできます。

この場合も、AIの提案をそのまま採用するのではなく、人間が確認します。

6.16 AIによる画像制作の支援

Project Sakuraでは、AIを画像制作の支援にも利用します。

記事によっては、文章だけでは説明しにくい内容があります。

その場合、

図解

↓

イメージ画像

↓

説明用画像

↓

アイキャッチ

などを用意します。

AIに画像のアイデアや構図を考えてもらったり、画像生成のための指示を作ったりすることができます。

ただし、画像についても、

記事内容と合っているか

↓

誤解を招かないか

↓

必要な情報が伝わるか

↓

ブログ全体の雰囲気合っているか

を確認します。

6.17 AIによるPython作成支援

Project Sakuraでは、Pythonによる自動化も進めていきます。

Pythonを学ぶ途中では、AIにプログラム作成を支援してもらうこともできます。

例えば、

「このフォルダにあるMarkdownファイルを一覧表示するPythonプログラムを作ってください」

という依頼ができます。

AIからコードが出てきたら、そのまま使うのではなく、

コードを読む

↓

何をしているか確認する

↓

実際に動かす

↓

結果を確認する

↓

必要なら修正する

という流れで進めます。

Project Sakuraでは、AIを使ってコードを書きながら、同時にPythonそのものも学んでいきます。

6.18 エラー調査にもAIを使う

プログラムを作っていると、エラーが発生します。

初心者の場合、エラーメッセージを見ても意味が分からないことがあります。

その場合、エラー内容をAIに説明してもらうことができます。

例えば、

「このエラーは何を意味しますか？」

「初心者向けに原因を説明してください」

「どこを確認すればよいですか？」

というように質問します。

ここでも、AIの回答を確認しながら、一つずつ原因を調べます。

エラーをなくすことだけが目的ではありません。

「なぜエラーが出たのか」を理解することも重要です。

6.19 AIに作業手順を整理してもらう

Project Sakuraでは、作業手順そのものも記録します。

例えば、

記事作成

↓

画像作成

↓

WordPress登録

↓

公開前確認

という作業があります。

AIに、

「この作業を初心者でも実行できる手順に整理してください」

と依頼することができます。

その結果を実際の作業で試します。

もし手順に問題があれば修正します。

つまり、

AIが手順案を作る

↓

実際に作業する

↓

問題を見つける

↓

手順を修正する

↓

正式な手順として保存する

という流れです。

これは第5章で説明した「作業記録」ともつながります。

6.20 AIとの会話をそのまま正式原稿にしない

Project Sakuraでは、チャットの内容をそのまま本にするではありません。

第1章でも、

原稿作成

↓

内容確認

↓

修正

↓

章の完成

↓

正式版へ反映

↓

GitHubへ保存

という流れを決めています。

AIとの会話も同じです。

AIとの会話

↓

必要な情報を整理

↓

原稿として書き直す

↓

確認

↓

完成

という工程を通します。

これによって、本書の内容を統一した構成で管理できます。

6.21 AIを使った記事制作の基本フロー

ここまでの内容を記事制作にまとめると、次のようになります。

企画

↓

AIにアイデア整理を依頼

↓

テーマ決定

↓

情報調査

↓

AIに情報整理を依頼

↓

記事構成作成

↓

AIに構成案を確認してもらう

↓

本文作成

↓

AIによる文章改善

↓

人間による内容確認

↓

画像制作

↓

記事と画像の最終確認

↓

WordPressへ登録

↓

公開

↓

アクセス確認

↓

改善

という流れです。

AIはこの中の複数の場所で利用できます。

しかし、すべてを自動的に実行するわけではありません。

6.22 人間による確認ポイント

Project Sakuraでは、AIを利用するほど「確認」が重要になります。

特に確認するポイントは、

目的に合っているか

↓

読者に合っているか

↓

情報が正しいか

↓

内容に不足がないか

↓

重複がないか

↓

表現に問題がないか

↓

画像が内容に合っているか

↓

公開して問題ないか

です。

AIが文章を作る速度が速くても、確認を省略してはいけません。

Project Sakuraでは、作成速度だけではなく、完成したものの品質も重視します。

6.23 AIに「確認役」をしてもらう

AIは文章を作るだけではありません。

確認役として利用することもできます。

例えば、完成した記事に対して、

「初心者が理解しにくい部分を指摘してください」

「説明が不足している部分を指摘してください」

「同じ内容を繰り返している部分を指摘してください」

と依頼します。

こうすることで、自分では気づかなかった問題を見つけるきっかけになります。

ただし、AIの指摘も最終判断ではありません。

AIの提案を見て、人間が必要性を判断します。

6.24 一つのAI回答だけを信じない

AIから回答を得た場合でも、重要な情報については確認します。

特に、

現在のサービス仕様

↓

料金

↓

法律

↓

規約

↓

製品仕様

↓

技術仕様

など、変更される可能性がある情報については注意が必要です。

Project Sakuraでは、AIの回答を「確認するための材料」として扱います。

必要に応じて公式情報や一次情報を確認し、記事に利用します。

6.25 AIに最新情報を確認させる場合の注意

AIに「最新情報を教えてください」と依頼しただけで、必ず最新情報が得られるとは限りません。

そのため、現在の情報が重要な記事では、

いつの情報なのか

↓

どこから得た情報なのか

↓

現在も有効なのか

を確認します。

Project Sakuraでは、記事公開前に情報の状態を確認することを基本とします。

特に、サービスの料金や機能、ソフトウェアの仕様などは変更される可能性があります。

6.26 AIの得意・不得意を理解する

AIを上手に利用するには、AIが何でもできると考えないことが重要です。

AIは、

文章を整理する

↓

案を複数出す

↓

説明を分かりやすくする

↓

コード作成を支援する

↓

問題点を指摘する

などに利用できます。

一方で、

Project Sakuraの最終的な目的

↓

実際の運営状況

↓

本当に公開してよいか

↓

最終的な責任

などをすべてAIに任せるわけではありません。

AIの能力を過大評価せず、道具として使います。

6.27 プロンプトを資産として考える

Project Sakuraでは、良いプロンプトができたら、その場限りで終わらせません。

例えば、

記事構成用プロンプト

↓

タイトル案作成用プロンプト

↓

文章確認用プロンプト

↓

画像制作支援用プロンプト

↓

Python作成支援用プロンプト

などを整理して保存できます。

同じ作業を何度も行う場合、毎回ゼロから指示を考える必要がなくなります。

これは、第5章で説明した「作業手順を記録する」という考え方にもつながります。

6.28 プロンプトを改善する

最初に作ったプロンプトが完璧である必要はありません。

実際に使って、

「この指示では情報が足りない」

「出力が長すぎる」

「初心者向けになっていない」

などの問題があれば修正します。

そして、

プロンプト v0.1

↓

改善

↓

プロンプト v0.2

↓

改善

↓

プロンプト v0.3

というように育てていきます。

これはProject Sakuraで大切にしている、

小さく始める

↓

実際に使う

↓

問題を見つける

↓

改善する

という考え方と同じです。

6.29 AIの使い方そのものを仕組み化する

Project Sakuraの最終目標は、AIを使うことそのものではありません。

AIを利用して、ブログ運営を継続できる仕組みを作ることです。

例えば、

記事テーマを整理する

↓

AIに構成案を作ってもら

↓

人間が確認する

↓

AIに文章作成を支援してもら

↓

人間が確認する

↓

画像を作る

↓

WordPressへ登録する

↓

公開後のデータを記録する

↓

次の記事へ反映する

という流れです。

この流れが安定したら、Pythonなどを利用して、一部の繰り返し作業を効率化できます。

6.30 AIとPythonの関係

AIとPythonは、Project Sakuraの中で別々のものではありません。

AIは、

考える

↓

整理する

↓

案を出す

↓

文章やコードを支援する

ために利用できます。

Pythonは、

ファイルを扱う

↓

データを処理する

↓

繰り返し作業を行う

↓

Webサービスと連携する

ために利用します。

将来的には、

AIが作業を支援する

↓

Pythonが定型処理を行う

という組み合わせも考えられます。

ただし、いきなり複雑な仕組みを作るではありません。

まずは、それぞれを小さく理解していきます。

6.31 AIとGitHubの関係

GitHubは、Project Sakuraのファイルや変更履歴を管理する場所です。

AIは、そのファイルを作ったり改善したりする作業を支援します。

例えば、

AIにMarkdown原稿の改善案を出してもらう

↓

人間が内容を確認する

↓

Chapter06.mdを修正する

↓

VS Codeで保存する

↓

GitHub Desktopで変更を確認する

↓

Commitする

↓

Pushする

という流れです。

このように、

AI

+

VS Code

+

GitHub Desktop

+

GitHub

を組み合わせることで、作成から保存までの流れを整理できます。

6.32 AIとWordPressの関係

Project Sakuraでは、最終的にWordPressへ記事を公開します。

AIは、

記事構成

↓

文章

↓

タイトル

↓

説明文

↓

画像制作の支援

などを担当できます。

一方、WordPressは、

記事を登録する

↓

公開する

↓

Web上で表示する

ための場所です。

将来的にはPythonなどを利用して、AIによる支援とWordPressの作業を連携させることも考えられます。

しかし、最初から完全自動化するではありません。

まず手作業で流れを理解し、その後に必要な部分を自動化します。

6.33 AIを使うときの基本ルール

Project Sakuraでは、AIを利用するときに次の基本ルールを設けます。

1. AIの回答をそのまま信じない
2. 重要な情報は確認する
3. AIが作った文章をそのまま正式原稿にしない
4. 目的を明確にしてからAIへ依頼する
5. AIと人間の役割を分ける
6. 分からないことはAIに説明してもらう
7. コードは実際に動かして確認する
8. 画像は記事内容と合っているか確認する
9. 良いプロンプトは保存する
10. 使った結果を見てプロンプトを改善する
11. 作業手順や失敗も記録する
12. 最終的な判断は人間が行う

このルールは、Project Sakuraが成長しても基本的な考え方として維持します。

6.34 AIを使うことで初心者でも進めやすくする

Project Sakuraは、プログラミング未経験者でも進められることを目標としています。

初心者にとって、

エラーが出た

↓

意味が分からない

↓

何を調べればよいか分からない

という状態は珍しくありません。

AIを利用すれば、

「このエラーを初心者向けに説明してください」

と質問できます。

また、

「次に何を確認すればよいですか」

「このコードは何をしていますか」

と質問できます。

このように、AIを学習支援にも利用します。

6.35 分からないことを質問できる環境を作る

Project Sakuraでは、分からないことを放置しません。

分からない

↓

質問する

↓

説明を読む

↓

実際に試す

↓

結果を確認する

↓

理解する

という流れを作ります。

AIは、この「質問する」部分を支援してくれます。

ただし、説明を読んだだけで終わらせません。

実際に操作して確認します。

Project Sakuraでは、

説明

↓

実際に操作

↓

結果を確認

↓

問題があれば修正

↓

次へ進む

という学び方を基本とします。

6.36 AIによる学習と実践をつなげる

AIに質問すると、いくらでも説明を受けることができます。

しかし、説明を読むだけでは、実際に使える知識にならないことがあります。

そのため、

説明を受ける

↓

小さな課題を実行する

↓

結果を見る

↓

分からない部分を質問する

↓

もう一度実行する

という流れを作ります。

例えばPythonなら、

AIにコードの説明をしてもらう

↓

VS Codeに入力する

↓

保存する

↓

実行する

↓

結果を確認する

という流れです。

Project Sakuraでは、AIを「答えを受け取る場所」ではなく、「実際に作るための支援役」として利用します。

6.37 AI活用の最終的な目的

Project SakuraでAIを使う最終的な目的は、

「AIを使えるようになること」

だけではありません。

目指しているのは、

AI

+

人間

+

Python

+

GitHub

+

WordPress

+

データ管理

を組み合わせ、ブログ運営を継続できる仕組みを作ることです。

AIはその中の重要な一部分です。

AIだけで完結するのではなく、他の仕組みとつながることでProject Sakura全体の価値が高まります。

6.38 第6章で決めた基本方針

第6章では、AI活用について次の基本方針を決めました。

1. AIを文章作成だけの道具として扱わない

2. AIをプロジェクト全体の支援役として利用する
3. AIと人間の役割を分ける
4. AIに目的を明確に伝える
5. 最初から完璧なプロンプトを作ろうとしない
6. AIの回答を確認する
7. 重要な情報は必要に応じて元情報を確認する
8. AIが作った文章をそのまま正式原稿にしない
9. AIを記事構成やタイトル案の作成に利用する
10. AIを文章改善や確認にも利用する
11. AIを画像制作の支援に利用する
12. AIをPython作成やエラー調査の支援に利用する
13. AIを作業手順の整理にも利用する
14. 良いプロンプトを保存する
15. 実際に使ってプロンプトを改善する
16. AIとPythonを将来的に組み合わせる
17. 最初から完全自動化しない
18. 実際の作業を理解してから自動化する
19. AIを学習支援にも利用する
20. 最終的な判断は人間が行う

6.39 第6章まとめ

Project Sakuraでは、AIを単なる文章作成ツールとして利用しません。

AIは、

アイデア整理

↓

情報整理

↓

記事構成

↓

文章作成

↓

タイトル作成

↓

画像制作

↓

Python作成

↓

エラー調査

↓

作業手順整理

など、さまざまな場面で利用します。

しかし、AIが作ったものをそのまま使用するわけではありません。

AIに支援してもらう

↓

人間が確認する

↓

必要なら修正する

↓

実際に使う

↓

結果を確認する

↓

改善する

という流れを基本とします。

また、良いプロンプトや作業手順ができたら、それを保存します。

そして、

使う

↓

記録する

↓

改善する

という流れを繰り返します。

これはProject Sakura全体の考え方である、

小さく始める

↓

実際に使う

↓

問題を見つける

↓

改善する

↓

必要な部分を自動化する

という考え方と同じです。

AIを使えば、作業の速度を上げられる可能性があります。

しかし、速く作ることだけが目的ではありません。

Project Sakuraが目指しているのは、AIを含めた複数の道具を組み合わせ、長期間続けられるブログ運営の仕組みを作ることです。

そのためには、

AIに何を任せるのか。

人間が何を判断するのか。

どこを確認するのか。

どの作業を記録するのか。

どこから自動化するのか。

を少しずつ整理していく必要があります。

第6章では、その基本的な考え方を整理しました。

これからProject Sakuraを実際に進める中で、AIの使い方もさらに具体化していきます。

Chapter Check

第6章を読み終えたら、次の質問を確認してみましょう。

Q1

Project Sakuraでは、AIを何のために利用しますか？

Q2

AIと人間の役割を分ける必要があるのはなぜですか？

Q3

AIに作ってもらった文章を、そのまま正式原稿にしないのはなぜですか？

Q4

AIに指示を出すとき、最初に明確にした方がよいものは何ですか？

Q5

プロンプトを改善するとき、どのような流れで進めますか？

Q6

AIを記事構成の作成に利用するメリットは何ですか？

Q7

AIが作った情報を確認する必要があるのはなぜですか？

Q8

AIはPythonの学習やエラー調査にどのように利用できますか？

Q9

良いプロンプトを保存するメリットは何ですか？

Q10

Project Sakuraでは、なぜ最初から完全自動化を目指さないのでしょうか？

Q11

AIとPythonは、Project Sakuraの中でどのように組み合わせられますか？

Q12

AIを利用するとき、最終的な判断を人間が行うのはなぜですか？

第7章への予告

第6章では、Project SakuraにおけるAIの役割を整理しました。

AIを、

「全部やってくれる道具」

としてではなく、

「作業と一緒に進める支援ツール」

として利用することが重要です。

また、

目的を決める

↓

AIへ依頼する

↓

結果を確認する

↓

修正する

↓

実際に使う

↓

改善する

という基本的な流れも確認しました。

第7章では、ここまで整理したAI活用を、実際の記事制作へさらに具体的につなげていきます。

どのような記事テーマを考えるのか。

AIにどのような情報整理を依頼するのか。

記事構成をどのように作るのか。

本文作成をどこまでAIに支援してもらうのか。

そして、人間はどの段階で確認するのか。

第7章では、

****「AIを活用した記事制作実践」****

をテーマとして、Project Sakuraの記事を実際に作るための具体的な方法へ進みます。

第6章で整理した「AIを支援役として使う」という考え方を、実際の作業に落とし込んでいきます。

Project Sakura 第1巻 基礎編 v0.1

第7章 AIを活用した記事制作実践

7.1 はじめに

第6章では、Project SakuraにおけるAIの役割について整理しました。

AIは、すべての作業を人間の代わりに行う道具ではありません。

Project Sakuraでは、

目的を決める

↓

AIへ依頼する

↓

結果を確認する

↓

必要な部分を修正する

↓

実際に使う

↓

改善する

という流れを基本とします。

第7章では、この考え方を実際の記事制作につなげます。

ここでは、記事を1本作るために必要な作業を、

テーマ決定

↓

読者設定

↓

情報整理

↓

記事構成

↓

タイトル

↓

本文

↓
確認
↓
画像
↓
公開準備
↓
公開後の改善

という流れに分けて考えます。

いきなり完成した記事を書こうとするのではなく、小さな作業に分けて一つずつ進めます。

これはProject Sakura全体の基本方針である「小さく始める」という考え方にもつながります。

7.2 記事制作を始める前に決めること

記事を書く前に、最低限決めておきたいことがあります。

それは、

- ・何について書くのか
- ・誰に向けて書くのか
- ・読者に何を伝えたいのか
- ・読者が記事を読んだあと、何ができるようになるのか

です。

例えば、テーマが「Python初心者向けの記事」だったとしても、それだけでは範囲が広すぎます。

「Pythonを勉強したい人」

だけではなく、

「プログラミング未経験で、Windowsパソコンを使ってPythonを初めて動かしたい人」

のように、読者を具体的にすると記事の方向が決めやすくなります。

Project Sakuraでは、記事を書くことそのものよりも、

「誰に、何を、どのように届けるのか」

を先に考えます。

7.3 記事テーマを決める

記事制作の最初の作業は、テーマを決めることです。

ただし、思いついたテーマをそのまま記事にする必要はありません。

まず候補を出します。

例えば、

- Python初心者向け
- GitHub初心者向け
- VS Codeの使い方
- AIを使った作業効率化
- ブログ運営の基本
- 画像制作の方法
- WordPressの基本操作

などです。

この段階では、候補をたくさん出しても構いません。

重要なのは、最初から1つに決めようとして時間を使いすぎないことです。

候補を出したあと、

「Project Sakuraの目的に合っているか」

「読者に役立つ内容か」

「実際に自分が説明できる内容か」

を確認します。

7.4 AIにテーマ候補を出してもらう

テーマ候補を考える作業では、AIを利用できます。

例えば、次のような依頼をします。

Project Sakuraの記事テーマを考えてください。

対象読者は、

プログラミング初心者です。

Project Sakuraでは、

AI、Python、GitHub、VS Code、WordPressなどを

少しずつ学びながらブログ運営を仕組み化します。

初心者が読みやすく、

実際に操作しながら学べる記事テーマを
10個提案してください。

各テーマについて、

- ・記事タイトル案
- ・読者
- ・記事で解決する問題

を簡単に説明してください。

ここで重要なのは、AIに「記事を書いて」とだけ頼まないことです。

目的や読者を伝えることで、Project Sakuraに合った提案を受けやすくなります。

AIから出てきた候補は、そのまま採用する必要はありません。

人間が確認して、

「これはProject Sakuraに合っている」

「これは今は必要ない」

と判断します。

7.5 テーマを1つに絞る

候補が出たら、その中から1つ選びます。

ここで確認するポイントは次のとおりです。

1. Project Sakuraとの関連性

Project Sakuraが目指している方向と関係があるかを確認します。

2. 読者の役に立つか

単に自分が書きたいだけでなく、読者が知りたい内容になっているかを考えます。

3. 自分が説明できるか

分からないことを無理に説明するのではなく、実際に確認しながら説明できるテーマを選びます。

4. 記事としてまとめられるか

範囲が広すぎるテーマは、複数の記事に分けることも考えます。

例えば、

「Pythonについて全部説明する」

では広すぎます。

「PythonをWindowsで初めて実行する方法」

のように、小さく分けることで初心者にも分かりやすくなります。

7.6 読者を具体的ににする

記事制作では、読者を具体的に考えることが重要です。

例えば、

「初心者向け」

だけでは、まだ少し曖昧です。

そこで、

- ・プログラミング未経験
- ・Windowsを使用
- ・専門用語が分からない
- ・コマンド入力に慣れていない
- ・実際に画面を見ながら進めたい

というように、読者の状態を整理します。

読者が具体的にになると、記事の説明方法も変わります。

専門用語を使った場合は説明を追加する。

操作手順ではクリックする場所を説明する。

エラーが出た場合の確認方法も書く。

このように、読者を考えることが記事全体の設計につながります。

7.7 読者の「困っていること」を考える

記事は、情報を並べるだけではありません。

読者が抱えている問題を解決するためのものです。

例えば、

「Pythonのインストール方法」

という記事なら、

「Pythonとは何か」

だけで終わるのではなく、

「インストールできたかどうかをどう確認するのか」

まで説明したほうが実用的です。

Project Sakuraでは、記事を考えるとき、

「読者は何に困っているのか」

を考えます。

そのうえで、

問題

↓

原因

↓

解決方法

↓

確認方法

↓

次にできること

という流れを作ります。

7.8 AIに情報を整理してもらう

テーマと読者が決まったら、次に情報を整理します。

この作業でもAIを利用できます。

例えば、

次の記事テーマについて、

記事を書く前の情報整理をしてください。

テーマ：

PythonをWindowsで初めて実行する方法

対象読者：

プログラミング未経験者

次の項目に分けて整理してください。

- ・読者が最初に知りたいこと
- ・事前に必要なもの

- ・説明すべき用語
- ・操作手順
- ・確認ポイント
- ・初心者がつまづきやすい点
- ・記事の最後に案内する次のステップ

まだ本文は書かないでください。

ここでは「本文を書かないでください」と指定しています。

これは重要です。

最初から本文まで作ってしまうと、情報整理と文章作成が混ざってしまいます。

Project Sakuraでは、できるだけ作業を分けます。

7.9 AIの回答を確認する

AIが情報を整理してくれたとしても、その内容をそのまま記事に使うわけではありません。

特に、

- ・ソフトウェアの仕様
- ・料金
- ・サービスの機能
- ・現在の操作方法
- ・設定項目
- ・外部サービスに関する情報

などは、実際の状況と一致しているか確認する必要があります。

AIの回答は、記事制作を進めるための材料として扱います。

必要な場合は公式サイトや実際の画面などを確認し、正しい情報に修正します。

Project Sakuraでは、

AIが答えた

=

確認完了

とは考えません。

7.10 記事構成を作る

情報整理が終わったら、記事の構成を作ります。

例えば、

PythonをWindowsで初めて実行する方法

はじめに

Pythonとは

事前に準備するもの

Pythonをインストールする

Pythonがインストールされたか確認する

最初のプログラムを作る

プログラムを実行する

エラーが出た場合

次に学ぶこと

まとめ

という形です。

この段階では、まだ本文を完成させる必要はありません。

まず「記事の骨組み」を作ります。

7.11 記事構成をAIに作ってもらう

記事構成についてもAIに支援してもらえます。

例えば、

以下の条件で初心者向けの記事構成を作ってください。

テーマ：

PythonをWindowsで初めて実行する方法

読者：

プログラミング未経験者

目的：

Pythonをインストールし、

最初のプログラムを実行できるようにする。

条件：

- ・初心者向け
- ・専門用語には説明を入れる
- ・操作手順を分かりやすくする
- ・各章の役割が分かる構成にする
- ・本文はまだ作成しない

このように、AIには「構成を作る」という仕事を依頼します。

7.12 記事構成を人間が確認する

AIが作った構成は、人間が確認します。

確認するポイントは、

- ・読者にとって順番が自然か
- ・説明が抜けていないか
- ・不要な項目がないか
- ・同じ内容を繰り返していないか
- ・最後まで読んだときに目的を達成できるか

です。

例えば、初心者向けの記事なのに、いきなり難しい専門用語が出てくる場合は順番を変更します。

AIが作った構成をそのまま採用するのではなく、Project Sakuraの記事として分かりやすい形に整えます。

7.13 タイトルを考える

構成が決まったら、タイトルを考えます。

タイトルは、記事の内容が分かることが重要です。

例えば、

「Pythonについて」

だけでは、何が分かる記事なのか分かりません。

一方、

「PythonをWindowsで初めて実行する方法 | 初心者向け」

なら、内容が分かりやすくなります。

タイトルを考えるときは、

- ・何についての記事なのか
- ・誰向けなのか
- ・何ができるようになるのか

を意識します。

7.14 AIにタイトル案を出してもらう

タイトル案もAIに作ってもらえます。

例えば、

以下の記事について、

初心者にも内容が分かりやすいタイトル案を
10個作ってください。

テーマ：

PythonをWindowsで初めて実行する方法

条件：

- ・内容を誇張しない
- ・初心者向けであることが分かる
- ・何を学べる記事なのか分かる
- ・大げさな表現を使わない

ここでも、候補を出してもらったあとに人間が選びます。

7.15 本文を一気に作らない

記事本文を作る段階では、最初から全文を書かせる方法もあります。

しかし、Project Sakuraでは、できるだけ小さく分けて作ります。

例えば、

1. はじめに
2. 基礎説明
3. 操作手順
4. 確認方法
5. トラブル対応
6. まとめ

というように分けます。

この方法なら、途中で内容がおかしくなった場合にも修正しやすくなります。

7.16 AIに本文を作ってもらおう

本文作成では、記事構成をAIに渡します。

例えば、

以下の記事構成に沿って、

「Pythonとは」の部分だけを書いてください。

対象読者：

プログラミング未経験者

条件：

- ・初心者にも分かる文章
- ・専門用語には簡単な説明を付ける
- ・必要以上に難しくしない
- ・内容を誇張しない
- ・この見出しの範囲だけを書く

このように範囲を限定します。

「記事を全部書いてください」

と依頼するよりも、

「この見出しだけを書いてください」

と依頼したほうが、確認しやすくなります。

7.17 AIが作った文章を確認する

本文が完成したら、人間が確認します。

確認するポイントは、

内容

事実関係に問題がないか確認します。

説明

初心者が読んで理解できるか確認します。

表現

不自然な文章や、同じ表現の繰り返しがないか確認します。

手順

実際の操作と文章が一致しているか確認します。

読者目線

「この説明だけで次の操作に進めるか」を考えます。

Project Sakuraでは、文章がきれいだから完成、とは考えません。

実際に読者が使える記事になっているかを確認します。

7.18 実際に操作して確認する

操作手順の記事では、特に実際の操作確認が重要です。

例えば、

「ここをクリックします」

と書いているなら、本当にその場所にボタンがあるか確認します。

「この画面が表示されます」

と書いているなら、実際に同じ画面が表示されるか確認します。

記事を書くことと、実際に操作することを分けないことが重要です。

Project Sakuraでは、

文章を書く

↓

実際に操作する

↓

違いを見つける

↓

文章を修正する

という流れを大切にします。

7.19 スクリーンショットを利用する

初心者向けの記事では、文章だけでは分かりにくい場合があります。

そのような場合は、スクリーンショットを利用します。

例えば、

- ・インストール画面
- ・設定画面
- ・ボタンの位置
- ・エラー画面
- ・成功したときの画面

などです。

ただし、画像を増やせばよいわけではありません。

読者が迷う場所に必要な画像を置きます。

画像には、

「何を見ればよいのか」

が分かる説明を付けます。

7.20 画像を作る

Project Sakuraでは、記事の内容に合わせて画像も作成します。

画像には、

- ・記事のアイキャッチ
- ・説明用画像
- ・操作手順用の画像
- ・図解
- ・比較画像

などがあります。

AIを画像制作の支援に利用することもできます。

ただし、画像についても、

「AIが作ったからそのまま公開する」

ではありません。

記事内容との一致、文字の誤り、見やすさ、必要性などを確認します。

7.21 記事と画像を一致させる

記事制作では、本文と画像の内容が一致していることが重要です。

例えば、本文では「ボタンAをクリック」と説明しているのに、画像では別のボタンを示していたら、読者は混乱します。

そのため、

本文

↓

画像

↓

実際の画面

の3つが一致しているか確認します。

画像は飾りではなく、読者の理解を助けるための材料として利用します。

7.22 記事の完成チェック

本文と画像ができたら、公開前チェックを行います。

チェック項目を作っておくと便利です。

内容チェック

- ☐ テーマから話が外れていない
- ☐ 読者が明確になっている
- ☐ 必要な説明が入っている
- ☐ 内容に誤りがないか確認した

操作チェック

- ☐ 実際に操作して確認した
- ☐ 手順の順番が正しい
- ☐ 画面説明が実際の画面と一致している

文章チェック

- ☐ 誤字脱字を確認した
- ☐ 不自然な表現がない
- ☐ 同じ説明を繰り返していない
- ☐ 初心者にも理解できる

画像チェック

- ☐ 本文と画像が一致している
- ☐ 画像が見やすい
- ☐ 不要な画像がない

公開チェック

- ☐ タイトルを確認した
- ☐ カテゴリを確認した
- ☐ 必要なタグを確認した
- ☐ 公開前に最終確認した

7.23 AIによる最終チェック

AIを最終確認の補助として利用することもできます。

例えば、

以下の記事を初心者向けの観点から確認してください。

確認してほしい項目：

- ・説明不足
- ・専門用語
- ・文章の分かりにくい部分
- ・手順の抜け
- ・同じ内容の繰り返し
- ・読者が途中で迷いそうな部分

記事の内容を勝手に書き換えず、
まず問題点だけを一覧にしてください。

ここでも、AIにいきなり全文を書き直させません。

まず問題点を出してもらいます。

その後、人間が修正する部分を判断します。

7.24 記事を公開する

記事の確認が終わったら、WordPressなどの公開環境へ移します。

この段階では、

タイトル

本文

画像

カテゴリ

タグ

公開設定

などを確認します。

最初からPythonで完全自動投稿する必要はありません。

まずは手作業で公開し、

「どの作業が繰り返し発生するのか」

を確認します。

繰り返し作業が見えてきたら、その作業を手順化し、将来的な自動化候補として記録します。

7.25 公開後も記事制作は終わらない

記事を公開したら終了ではありません。

Project Sakuraでは、

公開

↓

アクセス確認

↓

問題発見

↓

改善

↓

データ蓄積

という流れを作ります。

例えば、

- ・読者がどこで離れているか
- ・どの記事が読まれているか
- ・検索から訪問されているか
- ・どの記事へのアクセスが少ないか

などを確認します。

ただし、最初から大量のデータ分析を行う必要はありません。

まずは確認できるデータを少しずつ記録します。

7.26 記事を改善する

公開後に問題が見つかったら記事を改善します。

例えば、

「説明が長すぎる」

のであれば短くします。

「初心者には分かりにくい」

のであれば説明を追加します。

「画像が分かりにくい」

のであれば画像を修正します。

「必要な手順が抜けている」

のであれば手順を追加します。

この改善を繰り返すことで、記事そのものも成長していきます。

7.27 記事制作の記録を残す

Project Sakuraでは、記事そのものだけでなく、制作過程も重要な記録です。

例えば、

- どのテーマを選んだか
- なぜそのテーマにしたか
- どんなプロンプトを使ったか
- どこをAIに依頼したか
- どこを人間が修正したか
- どんな問題が発生したか
- どのように解決したか

などを記録します。

この記録が、将来的な改善材料になります。

また、同じ作業をするときに、過去の手順を再利用できます。

7.28 良いプロンプトを保存する

記事制作を繰り返していると、使いやすいプロンプトが見つかってきます。

例えば、

「初心者向けに記事構成を作るプロンプト」

「タイトル案を作るプロンプト」

「文章の問題点を確認するプロンプト」

などです。

これらを毎回チャットから探すのではなく、ファイルとして保存しておきます。

将来的には、

```
project-sakura
|
├─ prompts
|   └─ article_idea.md
|   └─ article_outline.md
|   └─ article_title.md
|   └─ article_check.md
```

のように整理できます。

これはProject Sakuraを「個人の作業」から「再利用できる仕組み」へ発展させるための重要な一歩です。

7.29 記事制作を手順化する

記事を何本か作ると、毎回同じ作業をしていることに気づきます。

例えば、

1. テーマを決める
2. 読者を決める
3. 情報を整理する
4. 構成を作る
5. タイトルを決める
6. 本文を作る
7. 確認する

8. 画像を作る
9. 公開する
10. 公開後に確認する

という流れです。

この手順をチェックリストにしておけば、毎回ゼロから考える必要がありません。

これが「手順化」です。

7.30 手順化から半自動化へ

手順化ができれば、次に「どこを自動化できるか」を考えます。

例えば、

記事テーマの一覧をファイルから読み込む。

AIへ渡すプロンプトをテンプレート化する。

記事の進行状況をCSVやJSONに記録する。

完成した記事のファイル名を自動的に整理する。

このような小さな自動化から始めます。

いきなり「ボタン1つでブログ記事を全部公開する」ことを目指しません。

Project Sakuraでは、

手作業

↓

手順化

↓

テンプレート化

↓

部分的な自動化

↓

連携

↓

必要に応じて高度な自動化

という順番で進めます。

7.31 Pythonが記事制作を支援する

Pythonは、記事そのものを書くためだけに使う必要はありません。

例えば、

- ファイル名の整理
- 記事一覧の管理
- データの集計
- テンプレート作成
- 定型処理
- 作業ログの保存

などに利用できます。

AIがPythonコードの作成を支援し、人間が実際に動かして確認します。

この組み合わせによって、

AI

+

Python

+

人間による確認

という仕組みを作ることができます。

7.32 記事制作とGitHub

Project Sakuraでは、記事制作に関するファイルもGitHubで管理します。

例えば、

project-sakura

|

├─ docs

|

├─ prompts

|

├─ articles

|

├─ images

|

└─ python

のように整理していくことができます。

記事原稿、プロンプト、プログラム、作業手順などを管理することで、プロジェクト全体の変更履歴を残せます。

例えば、

記事作成

↓

修正

↓

GitHubへ保存

↓

さらに修正

↓

GitHubへ保存

という形です。

これにより、「いつ、何を変更したのか」を確認できます。

7.33 記事制作で失敗した場合

記事制作では失敗することがあります。

例えば、

- ・テーマが広すぎた
- ・記事構成が分かりにくかった
- ・AIの回答を十分確認しなかった
- ・説明が難しすぎた
- ・画像と本文が一致していなかった
- ・公開後に修正点が見つかった

などです。

これらを「失敗したから消す」のではなく、Project Sakuraでは記録します。

例えば、

問題：

記事の説明が初心者には難しかった。

原因：

専門用語を説明せずに使用していた。

改善：

専門用語に簡単な説明を追加する。

次回：

記事構成を作る段階で専門用語を確認する。

という形です。

この記録が次の記事制作に役立ちます。

7.34 1本の記事を完成させるための基本フロー

ここまでの内容をまとめると、記事制作は次のようになります。

テーマ候補を出す



テーマを決める



読者を決める



読者の問題を整理する



AIに情報整理を依頼する



内容を確認する



記事構成を作る



AIに構成を確認してもらう



人間が構成を決定する



タイトルを決める



本文を作成する



内容を確認する



実際に操作して確認する



画像を作成する



本文と画像を確認する



公開前チェック



WordPressなどへ公開

↓

アクセスや反応を確認

↓

改善

↓

制作記録を保存

これが第7章で扱った記事制作の基本的な流れです。

7.35 AIに任せる作業と人間が行う作業

Project Sakuraでは、AIと人間の役割を分けます。

AIが得意な作業

- ・アイデアを大量に出す
- ・情報を整理する
- ・構成案を作る
- ・タイトル案を出す
- ・文章作成を支援する
- ・文章の問題点を探す
- ・プログラム作成を支援する
- ・作業手順を整理する

人間が行う作業

- ・Project Sakuraの目的を決める
- ・テーマを最終決定する
- ・情報の正しさを確認する
- ・実際の操作を確認する
- ・記事を公開するか判断する
- ・改善内容を決める

AIと人間が同じ仕事をするのではなく、それぞれの得意な部分を組み合わせます。

7.36 初心者の場合は「完成」を小さくする

初心者の場合、最初から完璧な記事を作ろうとすると作業量が大きくなります。

そこで、

「今日はテーマを決める」

「次に構成を作る」

「その次に本文を書く」

というように作業を分けます。

Project Sakuraでは、小さな完成を積み重ねることを重視します。

例えば、

テーマ決定 完成

↓

構成完成

↓

本文完成

↓

画像完成

↓

公開準備完成

↓

公開完成

↓

改善完成

という形です。

1回の作業量を小さくすることで、継続しやすくなります。

7.37 記事制作で大切にすること

第7章では、特に次の5つを大切にします。

1. 読者を先に考える

自分が書きたい内容だけではなく、読者が何を必要としているかを考えます。

2. AIに作業を分けて依頼する

テーマ、構成、本文、確認などを分けて依頼します。

3. AIの結果を確認する

AIが作った情報を、そのまま正しいと判断しません。

4. 実際に使って確認する

操作手順や説明は、実際に確認します。

5. 作業を記録する

うまくいった方法も、失敗した方法も記録します。

この5つを続けることで、記事制作そのものを改善できます。

7.38 Project Sakuraの記事制作は「作って終わり」ではない

Project Sakuraでは、記事を公開したら終わりではありません。

記事を作る。

公開する。

結果を見る。

改善する。

その経験を次の記事に利用する。

この循環を作ることが重要です。

つまり、

記事A

↓

公開

↓

確認

↓

改善

↓

学習

↓

記事B

↓

公開

↓

確認

↓

改善

↓

学習

↓

記事C

という流れを作ります。

記事が増えるだけでなく、記事制作の方法そのものも改善されていきます。

7.39 記事制作の仕組みを育てる

最初は、すべて手作業でも構いません。

むしろ、最初は手作業で行ったほうが、どの作業が必要なのかを理解できます。

例えば、

最初：

手作業で記事を書く

↓

次：

記事制作チェックリストを作る

↓

次：

プロンプトをテンプレート化する

↓

次：

Pythonでファイル整理を自動化する

↓

次：

データ管理を自動化する

↓

次：

必要な部分だけ外部サービスと連携する

というように進めます。

自動化すること自体を目的にしません。

「作業を継続しやすくする」

ことが目的です。

7.40 記事制作の成果物

1本の記事を作るとき、完成するのは記事本文だけではありません。

例えば、

記事本文

記事タイトル

記事構成

画像

プロンプト

チェックリスト

作業記録

改善記録

などが成果物になります。

これらを整理して保存することで、次の記事制作に再利用できます。

Project Sakuraでは、このような「再利用できるもの」を少しずつ増やしていきます。

7.41 第7章まとめ

第7章では、AIを活用して記事を制作する具体的な流れを整理しました。

記事制作は、

テーマ決定

↓

読者設定

↓

情報整理

↓

構成作成

↓

タイトル作成

↓

本文作成

↓

確認

↓

画像制作

↓

公開

↓

改善

という複数の作業に分けることができます。

AIは、それぞれの作業を支援できます。

しかし、AIにすべてを任せるではありません。

AIが提案する。

人間が確認する。

実際に使ってみる。

必要なら修正する。

そして、作業方法そのものも記録します。

この流れを繰り返すことで、Project Sakuraの記事制作は少しずつ改善されます。

また、記事制作の作業を手順化し、プロンプトやチェックリストを保存することで、将来的な自動化につなげることもできます。

Project Sakuraでは、

「AIに記事を書かせる」

ことだけを目標にはしません。

目指しているのは、

「AIを活用しながら、継続的に記事を作り、改善できる仕組み」

です。

Chapter Check

第7章を読み終えたら、次の質問を確認してみましょう。

Q1

記事制作を始める前に、なぜ「誰に、何を届けるのか」を考えるのでしょうか？

Q2

AIに記事テーマの候補を出してもらうとき、どのような情報を伝えるとよいでしょうか？

Q3

AIに情報整理を依頼するとき、最初から本文まで作らせない方法にはどのようなメリットがありますか？

Q4

記事構成を作る目的は何ですか？

Q5

AIが作った記事構成を、そのまま使用せず人間が確認するのはなぜですか？

Q6

AIに本文を書いてもらうとき、記事全体ではなく一つの見出しごとに依頼するメリットは何ですか？

Q7

操作手順の記事で、実際に操作して確認することが重要なのはなぜですか？

Q8

記事と画像の内容を一致させる必要があるのはなぜですか？

Q9

公開後も記事を改善するのはなぜですか？

Q10

記事制作で使用した良いプロンプトを保存するメリットは何ですか？

Q11

Project Sakuraでは、なぜ最初から記事制作を完全自動化しないのでしょうか？

Q12

記事制作におけるAIと人間の役割を、それぞれ説明してください。

第8章への予告

第7章では、AIを活用した記事制作の具体的な流れを整理しました。

テーマを決める。

読者を考える。

情報を整理する。

記事構成を作る。

本文を書く。

確認する。

画像を作る。

公開する。

そして、公開後に改善する。

ここまでで、Project Sakuraの記事制作の基本的な流れが見えてきました。

第8章では、この流れをさらに一歩進めます。

記事を継続的に作っていくためには、

「記事を作ったあと、どのように管理するのか」

という問題が出てきます。

記事のタイトル。

公開状態。

カテゴリ。

作成日。

更新日。

使用したプロンプト。

画像。

アクセス状況。

改善履歴。

これらを毎回バラバラに管理すると、記事数が増えたときに分かりにくくなります。

そこで第8章では、

****「記事データ管理と制作記録」****

をテーマとして、Project Sakuraの記事や制作情報をどのように整理していくのかを考えます。

そして、そのデータを将来的にPythonやAIによる自動処理へつなげるための基礎を作ります。

第7章で「記事を作る方法」を整理し、第8章では「作った記事を管理する方法」へ進みます。

Project Sakura 第1巻 基礎編 v0.1

第8章 記事データ管理と制作記録

8.1 はじめに

第7章では、AIを活用して1本の記事を制作する流れを整理しました。

テーマを決める。

読者を考える。

情報を整理する。

記事構成を作る。

本文を書く。

画像を作る。

確認する。

公開する。

そして、公開後に改善する。

記事を1本作るだけなら、この流れを手作業で管理しても大きな問題はありません。

しかし、記事が2本、3本、10本、50本と増えていくと、別の問題が出てきます。

「この記事は今どこまで完成しているのか」

「このタイトルは確定しているのか」

「いつ公開したのか」

「どのプロンプトを使ったのか」

「どの画像を使ったのか」

「最後にいつ更新したのか」

「どこを改善したのか」

といった情報を、頭の中だけで管理することが難しくなります。

そこで第8章では、記事そのもののだけでなく、記事に関する情報を整理して管理する方法を考えます。

Project Sakuraでは、記事を作る仕組みだけではなく、

「作った記事を管理できる仕組み」

も少しずつ作っていきます。

8.2 なぜ記事データを管理するのか

記事が増えると、ファイルの数も増えます。

例えば、

記事01

記事02

記事03

記事04

記事05

という状態になったとします。

ファイル名だけでは、それぞれの記事がどのような状態なのか分からない場合があります。

そこで、

- 記事タイトル
- 記事ID
- カテゴリ
- 作成日
- 更新日
- 公開状態
- 使用画像
- 使用プロンプト
- 公開URL
- 改善履歴

などを記録します。

これらの情報を「記事データ」として扱います。

記事データを整理しておくことで、記事数が増えても現在の状況を把握しやすくなります。

8.3 記事そのものと記事データを分けて考える

初心者のうちは、

「記事」と「記事に関する情報」は同じものに見えるかもしれません。

しかし、Project Sakuraでは分けて考えます。

例えば記事本文が、

PythonをWindowsで初めて実行する方法

だったとします。

その記事には、

タイトル

カテゴリ

作成日

更新日

公開状態

URL

画像

プロンプト

メモ

などの情報があります。

記事本文そのものと、それを管理するための情報を分けて考えることで、後からPythonなどで処理しやすくなります。

8.4 記事にIDを付ける

記事を管理するとき、記事ごとに識別できるIDを付ける方法があります。

例えば、

ART-0001

ART-0002

ART-0003

のような形式です。

タイトルは後から変更される可能性があります。

例えば、

「Python初心者向け入門」

というタイトルを、

「PythonをWindowsで初めて実行する方法」

に変更することがあります。

しかし、記事IDを変えなければ、同じ記事として管理できます。

そのため、記事タイトルとは別に、管理用のIDを持たせる方法が便利です。

8.5 記事データの基本項目

最初から大量の項目を作る必要はありません。

まずは必要な項目から始めます。

例えば、

記事ID

タイトル

カテゴリ

ステータス

作成日

更新日

公開日

URL

などです。

Project Sakuraでは、実際に記事を作りながら、必要な項目を追加していきます。

最初から完璧なデータベースを設計する必要はありません。

8.6 記事のステータスを管理する

記事制作では、記事がどの状態なのかを管理することが重要です。

例えば、

企画中

↓

構成作成中

↓

執筆中

↓

確認中

↓

画像作成中

↓

公開準備

↓

公開済み

↓

改善中

という状態があります。

これを「ステータス」として管理します。

例えば、

status = planning

なら企画中、

status = writing

なら執筆中、

status = published

なら公開済み、

というように管理できます。

初心者のうちは日本語で管理しても構いません。

重要なのは、「今どの状態なのか」が分かることです。

8.7 記事制作の進捗を見えるようにする

ステータスを管理すると、記事制作の進み具合が分かります。

例えば、

記事01 公開済み

記事02 確認中

記事03 執筆中

記事04 構成作成中

記事05 企画中

という状態です。

これを見るだけで、

「次に何をすればよいのか」

が分かります。

記事数が増えたときには、この進捗管理が非常に重要になります。

8.8 記事データをMarkdownで管理する

Project Sakuraでは、最初から専用のデータベースを用意する必要はありません。

Markdownファイルでも情報を整理できます。

例えば、

記事情報

- 記事ID：ART-0001
- タイトル：PythonをWindowsで初めて実行する方法
- カテゴリ：Python
- ステータス：公開済み
- 作成日：2026-08-01
- 更新日：2026-08-05
- 公開日：2026-08-06

のように記録できます。

人間が読んでも分かりやすく、GitHubでも管理できます。

8.9 CSVという方法

記事データを表形式で管理する方法もあります。

その代表的な形式がCSVです。

CSVは、

Comma-Separated Values

の略です。

簡単に言えば、表のデータを文字として保存する形式です。

例えば、

記事ID,タイトル,カテゴリ,ステータス

ART-0001,PythonをWindowsで初めて実行する方法,Python,公開済み

ART-0002,GitHub初心者向け入門,GitHub,執筆中

のように保存できます。

Excelなどの表計算ソフトでも扱えます。

8.10 JSONという方法

Pythonなどのプログラムで扱いやすい形式として、JSONがあります。

例えば、

```
{  
  "id": "ART-0001",  
  "title": "PythonをWindowsで初めて実行する方法",  
  "category": "Python",  
  "status": "published"  
}
```

のように記録できます。

JSONは、人間が読めるだけでなく、プログラムでも扱いやすい形式です。

Project Sakuraでは、将来的にPythonを使って記事データを処理するため、JSONも重要な候補になります。

8.11 Markdown・CSV・JSONの違い

それぞれに向いている用途があります。

Markdown

人間が読むための文章や説明に向いています。

CSV

表形式のデータ管理に向いています。

JSON

プログラムから扱うデータに向いています。

例えば、

説明書 → Markdown

記事一覧 → CSV

プログラム用データ → JSON

という使い分けが考えられます。

ただし、最初からすべての形式を使う必要はありません。

Project Sakuraでは、実際の用途を確認しながら選択します。

8.12 記事ファイルの整理

記事本文も、ルールを決めて保存すると管理しやすくなります。

例えば、

articles

```
|
|— ART-0001
|   |— article.md
|   |— images
|   └─ notes.md
|
|— ART-0002
|   |— article.md
|   |— images
|   └─ notes.md
|
└─ ART-0003
    |— article.md
    |— images
    └─ notes.md
```

という構成です。

こうすると、記事ごとの関連ファイルをまとめて管理できます。

8.13 記事と画像を関連付ける

記事に画像を使う場合、どの画像がどの記事のものなのか分かるようにします。

例えば、

```
ART-0001-main.png
ART-0001-step01.png
ART-0001-step02.png
```

のように名前を付けます。

このように記事IDをファイル名に含めると、後から探しやすくなります。

画像が増えたときにも整理しやすくなります。

8.14 プロンプトの記事と関連付ける

第7章では、記事制作に使用したプロンプトを保存することについて説明しました。

第8章では、さらに記事とプロンプトを関連付けます。

例えば、

```
ART-0001
↓
article_idea.md
```


article_outline.md

article_title.md

article_check.md

のようにします。

こうすると、

「この記事はどのプロンプトを使って作ったのか」

を後から確認できます。

8.15 制作記録を残す

記事制作では、完成した記事だけが成果物ではありません。

制作途中の記録も重要です。

例えば、

2026-08-01

テーマ候補を作成

2026-08-02

テーマ決定

2026-08-03

記事構成完成

2026-08-04

本文完成

2026-08-05

画像作成

2026-08-06

公開

という記録です。

このような記録を残しておけば、

「記事を1本作るのにどれくらい時間がかかったか」

「どの作業に時間がかかったか」

なども後から確認できます。

8.16 作業時間を記録する

記事制作では、作業時間を記録する方法もあります。

例えば、

テーマ決定	20分
構成作成	30分
本文作成	60分
画像作成	30分
確認	20分

という形です。

これを何本か記録すると、

「どの作業に時間がかかるのか」

が見えてきます。

時間がかかっている作業が分かれば、そこを改善できます。

8.17 失敗記録もデータになる

Project Sakuraでは、失敗も記録します。

例えば、

問題：

記事構成が広すぎて本文が長くなった。

原因：

最初に扱う範囲を決めていなかった。

改善：

記事の目的を1つに絞る。

次回：

構成作成前に「この記事で読者ができるようになること」を1つ決める。

このような記録も記事データの一部として扱えます。

失敗を記録すると、次の記事で同じ問題を避けられます。

8.18 改善履歴を残す

公開後に記事を修正した場合も、その内容を記録します。

例えば、

2026-08-10

見出し2を追加

理由：

初心者向けの説明が不足していたため。

2026-08-12

画像を差し替え

理由：

操作画面が古くなっていたため。

という形です。

これにより、記事がどのように成長してきたのか分かります。

8.19 GitHubと記事管理

Project Sakuraでは、GitHubが変更履歴を管理する場所として利用されています。

記事データについても、GitHubを活用できます。

例えば、

記事作成

↓

article.mdを保存

↓

GitHubへCommit

↓

記事を修正

↓

再度Commit

という流れです。

これにより、

「いつ、どんな変更をしたのか」

を確認できます。

8.20 GitHubに保存するもの

Project Sakuraでは、次のようなものをGitHubで管理できます。

記事原稿

プロンプト

Pythonプログラム

設計書

チェックリスト

制作記録

改善記録

README

一方で、すべてのデータを無条件にGitHubへ保存するわけではありません。

パスワードやAPIキーなどの秘密情報は、公開リポジトリに保存してはいけません。

今後外部サービスと連携するときには、秘密情報の管理方法も別途学びます。

8.21 READMEを活用する

フォルダに何が入っているのかを説明するために、READMEを利用できます。

例えば、

```
# articles
```

Project Sakuraの記事原稿を管理するフォルダです。

```
## 構成
```

```
- ART-0001 : Python関連記事
```

```
- ART-0002 : GitHub関連記事
```

```
- ART-0003 : AI関連記事
```

のように書きます。

READMEは、後から自分が見返したときにも役立ちます。

8.22 記事一覧を作る

記事が増えたら、記事一覧を作ります。

例えば、

ID	タイトル	カテゴリ	状態
ART-0001	PythonをWindowsで初めて実行する方法	Python	公開済み
ART-0002	GitHub初心者向け入門	GitHub	執筆中
ART-0003	AIを使った記事制作	AI	構成作成中

という表です。

この一覧を見ることで、全体の状況を一度に確認できます。

8.23 記事一覧を手作業で作る

最初は手作業で十分です。

ExcelやCSV、Markdownの表などで管理できます。

重要なのは、

「現在の状態を見えるようにする」

ことです。

最初からPythonで自動生成する必要はありません。

まず手作業で管理してみます。

8.24 手作業から自動化へ

記事数が増えてきたら、

「毎回同じ情報を入力している」

ということに気づきます。

例えば、

記事ID

タイトル

カテゴリ

ステータス

作成日

などです。

このような定型作業は、将来的にPythonで自動化できます。

例えば、

「新しい記事を登録する」

という操作をすると、自動的に、

- 記事IDを作る
- 記事フォルダを作る
- 初期ファイルを作る
- 記事一覧を更新する

という処理を行う仕組みです。

8.25 Pythonで記事データを扱う

Pythonでは、CSVやJSONなどのデータを読み込んだり、書き込んだりできます。

例えば、記事一覧を読み込んで、

公開済みの記事だけ表示する

という処理を作ることができます。

また、

執筆中の記事を一覧表示する

こともできます。

さらに、

一定期間更新されていない記事を探す

といった処理にも発展できます。

これが記事データを「保存する」段階から「活用する」段階への変化です。

8.26 記事データをAIと組み合わせる

記事データが整理されていれば、AIと組み合わせることもできます。

例えば、

「現在、執筆中の記事を一覧にしてください」

「公開済みだが長期間更新していない記事を確認してください」

「Pythonカテゴリの記事に不足しているテーマを考えてください」

といった支援につながられます。

ただし、AIにデータを渡す場合も、どのデータを使うのかを明確にする必要があります。

データが整理されていなければ、AIに渡す情報も整理できません。

そのため、AI活用の前提としてデータ管理が重要になります。

8.27 記事制作の「状態」をデータにする

第7章では、記事制作を手順として考えました。

第8章では、その状態をデータとして扱います。

例えば、

planning

outlining

writing

review

image

ready

published

improving

という状態です。

これを記事データに保存します。

すると、プログラムから、

「statusがwritingの記事だけ取り出す」

ということができるようになります。

8.28 記事管理の最初のルール

Project Sakuraでは、記事管理について最初にいくつかのルールを決めます。

ルール1

記事には管理用IDを付ける。

ルール2

記事の状態を記録する。

ルール3

作成日と更新日を記録する。

ルール4

関連する画像やプロンプトを分かるようにする。

ルール5

重要な変更や失敗を記録する。

ルール6

後からPythonで扱える形を意識する。

これらを最初の基本ルールとします。

8.29 最初から複雑にしない

記事管理を始めると、

「もっと細かく管理したほうがいいのではないか」

と考えたくなります。

しかし、項目を増やしすぎると、入力作業そのものが大変になります。

例えば、最初から30項目を入力する仕組みにすると、記事を作るたびに大量の入力が必要になります。

その結果、管理すること自体が負担になる可能性があります。

Project Sakuraでは、

必要な項目だけ作る

↓

実際に使う

↓

問題を見つける

↓

必要なら項目を追加する

という順番にします。

8.30 記事データを育てる

記事管理の仕組みも、記事と同じように少しずつ改善します。

最初：

ID

タイトル

状態

次に、

ID

タイトル

カテゴリ

状態

作成日

更新日

さらに、

URL

画像

プロンプト

改善履歴

を追加するかもしれません。

このように、実際の運用に合わせてデータ項目を増やします。

8.31 記事管理とブログ運営

記事データを管理すると、ブログ全体の状況も見やすくなります。

例えば、

公開済み 20記事

執筆中 3記事

構成作成中 2記事

企画中 5記事

というように把握できます。

さらにカテゴリ別に、

Python 10記事

AI 8記事

GitHub 5記事

ブログ 7記事

という集計もできます。

これにより、

「今、どの分野の記事が多いのか」

を確認できます。

8.32 データから次の記事を考える

記事データが蓄積されると、新しい記事を考える材料にもなります。

例えば、Pythonの記事が多いのにGitHubの記事が少ないことが分かった場合、

「GitHub初心者向けの記事を増やす」

という判断ができます。

また、ある記事に関連する記事が不足している場合、

「関連記事を作る」

という考え方もできます。

つまり、データ管理は単なる整理ではありません。

次の行動を決めるための材料にもなります。

8.33 アクセスデータとの関係

将来的には、記事の制作データだけでなく、公開後のデータも管理します。

例えば、

- ・アクセス数
- ・検索からの訪問
- ・クリック
- ・記事ごとの反応
- ・更新日時

などです。

ただし、これらを第8章ですべて自動取得する必要はありません。

まずは、

「記事制作データ」

を整理することから始めます。

公開後のアクセスデータについては、今後の章で段階的に扱います。

8.34 データを蓄積する意味

Project Sakuraの最終目標は、単発の記事制作ではありません。

記事を作る。

公開する。

データを残す。

改善する。

次の記事を作る。

この循環を繰り返します。

そのためには、過去の情報が残っている必要があります。

データを蓄積することで、

「以前どうしたのか」

「何がうまくいったのか」

「何が問題だったのか」

を確認できます。

8.35 記事制作の再現性を高める

記事制作を仕組み化する目的の一つが「再現性」です。

例えば、以前の記事を作るときに、

- ・テーマ決定
- ・構成
- ・プロンプト
- ・チェック方法
- ・画像制作
- ・公開手順

を記録しておけば、次の記事でも同じ手順を利用できます。

毎回ゼロから考える必要がなくなります。

これが、作業を「仕組み」に変えていくということです。

8.36 記事管理の実践例

ここまでの内容を、簡単な例でまとめます。

記事ID：ART-0001

タイトル：

PythonをWindowsで初めて実行する方法

カテゴリ：

Python

ステータス：

published

作成日：

2026-08-01

更新日：

2026-08-05

使用画像：

ART-0001-main.png

ART-0001-step01.png

使用プロンプト：

article_outline.md

article_title.md

article_check.md

メモ：

初心者向けに説明を追加した。

このように情報を整理しておけば、後から記事の状態を確認できます。

8.37 記事制作記録の実践例

記事そのものとは別に、制作記録を残すこともできます。

2026-08-01

テーマ候補を10個作成。

2026-08-02

Pythonテーマを採用。

2026-08-03

AIを利用して構成案を作成。

2026-08-04

構成を修正。

2026-08-05

本文完成。

2026-08-06

実際の操作を確認。

2026-08-06

画像作成。

2026-08-07

公開。

2026-08-10

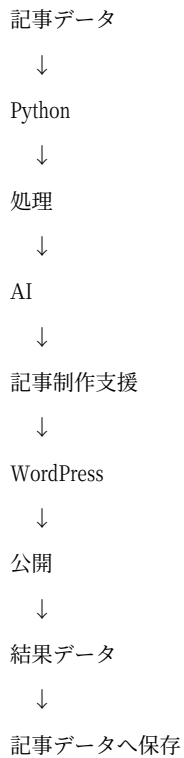
初心者向け説明を追加。

この記録は、次の記事制作にも利用できます。

8.38 記事データ管理を将来の自動化につなげる

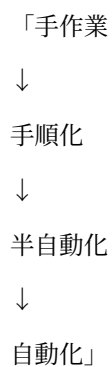
第8章で整理した記事データは、今後の自動化の基礎になります。

例えば将来的に、



という流れを作ることができます。

これは第1章で説明した、



という発展にもつながります。

8.39 自動化する前に手作業で理解する

ここで重要なのが、自動化を急がないことです。

例えば、

「記事登録をPythonで自動化したい」

と思ったとしても、まず手作業で記事を登録してみます。

その中で、

「毎回同じ入力をしている」

「同じフォルダを作っている」

「同じファイルを作っている」

という作業を見つけます。

そこから自動化します。

自動化する対象を理解せずにプログラムを作ると、必要のない機能まで作ってしまう可能性があります。

8.40 初心者が最初に作る管理表

最初は非常に簡単な表で構いません。

例えば、

ID	タイトル	状態
ART-0001	PythonをWindowsで初めて実行する方法	公開済み
ART-0002	GitHub初心者向け入門	執筆中
ART-0003	AIを使った記事制作	構成作成中

これだけでも記事の状況が分かります。

次に必要になったら、

カテゴリ

作成日

更新日

URL

などを追加します。

8.41 記事管理の最初の目標

第8章での最初の目標は、複雑な管理システムを作ることではありません。

まず、

「自分が作っている記事を一覧で確認できる」

ことを目標にします。

そして、

「今日の記事を進めるべきか」

「どの記事を修正するべきか」

が分かる状態を作ります。

それだけでも、記事制作の管理は大きく変わります。

8.42 第8章まとめ

第8章では、記事データ管理と制作記録について整理しました。

記事が増えると、

「どの記事があるのか」

だけではなく、

「どの状態なのか」

「いつ作ったのか」

「いつ更新したのか」

「どんなプロンプトを使ったのか」

「どんな問題があったのか」

などの情報も重要になります。

そのため、記事本文と記事に関する情報を分けて管理します。

記事IDを付ける。

ステータスを記録する。

作成日と更新日を記録する。

画像やプロンプトを関連付ける。

制作記録を残す。

改善履歴を残す。

これらを少しずつ積み重ねます。

データ管理には、

Markdown

CSV

JSON

などの方法があります。

最初からすべてを使う必要はありません。

人間が読みやすい方法から始め、必要になったらPythonで扱いやすい形式へ発展させます。

そして、整理された記事データは、将来的なAI活用やPythonによる自動化の基礎になります。

Project Sakuraでは、

記事を作る

↓

記事を公開する

↓

記事データを残す

↓

結果を確認する

↓

改善する

↓

次の記事を作る

という循環を作っていきます。

記事そのものだけでなく、

「記事を作った記録」

「記事を改善した記録」

「作業方法を改善した記録」

も資産として残します。

これが、Project Sakuraにおける記事データ管理の基本的な考え方です。

Chapter Check

第8章を読み終えたら、次の質問を確認してみましょう。

Q1

記事数が増えたとき、記事データを管理する必要があるのはなぜですか？

Q2

記事本文と記事データを分けて考えるメリットは何ですか？

Q3

記事IDを付ける理由は何ですか？

Q4

記事の「ステータス」とは何ですか？

Q5

Markdown、CSV、JSONは、それぞれどのような用途に向いていますか？

Q6

記事制作の記録を残すことには、どのようなメリットがありますか？

Q7

失敗や改善履歴も記録するのはなぜですか？

Q8

記事データをPythonで扱えるようにすると、どのようなことができるようになりますか？

Q9

なぜ最初から複雑な記事管理システムを作らないのでしょうか？

Q10

記事データを蓄積することは、次の記事制作にどのように役立ちますか？

Q11

Project Sakuraで、自動化する前に手作業で作業を確認する理由は何ですか？

Q12

第8章で学んだ記事データ管理を、将来どのような自動化につながられるのでしょうか？

第9章への予告

第8章では、記事を作った後に必要となる記事データの管理について整理しました。

記事ID。

ステータス。

作成日。

更新日。

画像。

プロンプト。

制作記録。

改善履歴。

これらを整理することで、記事が増えても全体の状況を把握しやすくなります。

しかし、記事データを手作業で管理していると、次第に同じ作業が増えてきます。

例えば、

「新しい記事を登録する」

「記事IDを付ける」

「フォルダを作る」

「初期ファイルを作る」

「一覧を更新する」

といった作業です。

ここで、Pythonが役立ちます。

第9章では、

****「Pythonによる記事データ管理」****

をテーマとして、実際に小さなプログラムを作りながら記事データを扱っていきます。

最初から複雑なシステムは作りません。

まずは、

データを保存する

↓

データを読み込む

↓

データを表示する

↓

データを追加する

↓

データを更新する

という基本的な処理から始めます。

第8章で「何を管理するのか」を整理し、第9章では「Pythonでどう扱うのか」へ進みます。

Project Sakuraの自動化は、ここから少しずつプログラムとして形になっていきます。